JavaScript

COMPLETE NOTES

CodeWithDeveeswar

Module 1 - JavaScript Introduction

1. What is JavaScript?

JavaScript is a high-level, interpreted scripting language as well as a programming language used to create dynamic and interactive web pages. It is one of the core technologies of the World Wide Web.

2. What is the history of JavaScript?

- JavaScript was created by Brendan Eich in 1995.
- It was developed at Netscape Communications Corporation.
- It was initially called Mocha, then LiveScript, and finally JavaScript.
- JavaScript was developed in approximately 10 days.
- It was created to make web pages dynamic and interactive.
- JavaScript was first released in December 1995.

3. What are the different versions of JavaScript (ECMAScript)?

- JavaScript was standardized by ECMA International as ECMAScript (ES).

Year	Version	Milestone
1997	ECMAScript 1 (ES1)	ECMA standard established.
1998	ECMAScript 2 (ES2)	Second edition of the ECMAScript standard.
1999	ECMAScript 3 (ES3)	Better error handling and browser improvements.
2003	ECMAScript 4 (ES4)	Planned major updates; ES4 was later abandoned.
2000–2005	—	AJAX / XMLHttpRequest became popular for web applications.
2009	ECMAScript 5 (ES5)	JSON support and improved JavaScript features.
2015	ECMAScript 6 (ES6)	Major modern updates with cleaner syntax (syntactic sugar).
2016–2021	ES7 to ES12	New ECMAScript versions released yearly.

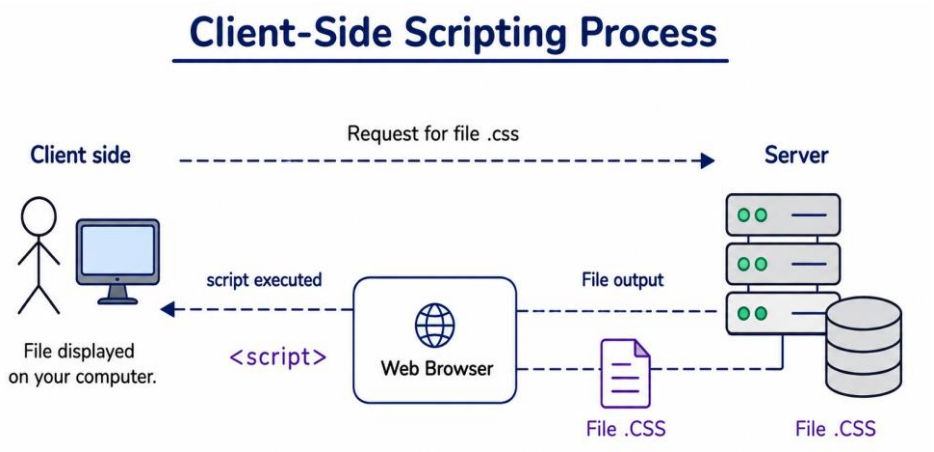
Module 2 - JavaScript Features & Characteristics

1. What are the features and characteristics of JavaScript?

- JS is mainly used for Client-side scripting
- Interpreted programming language
- Dynamically typed language
- Asynchronous programming language
- Event-driven programming Model
- Single Page Applications (SPA)
- Object-based scripting language
- Platform independent

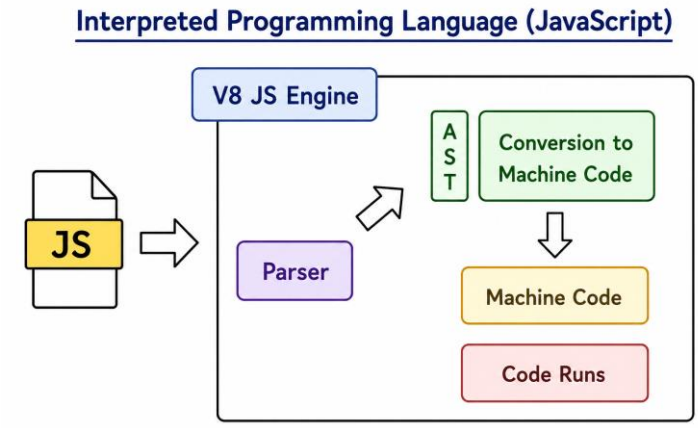
2. Client-Side Scripting

JavaScript code is executed directly in the user's web browser to make web pages interactive and dynamic.



3. Interpreted Programming Language

JavaScript code is executed line by line by the V8 Engine or JavaScript engine.

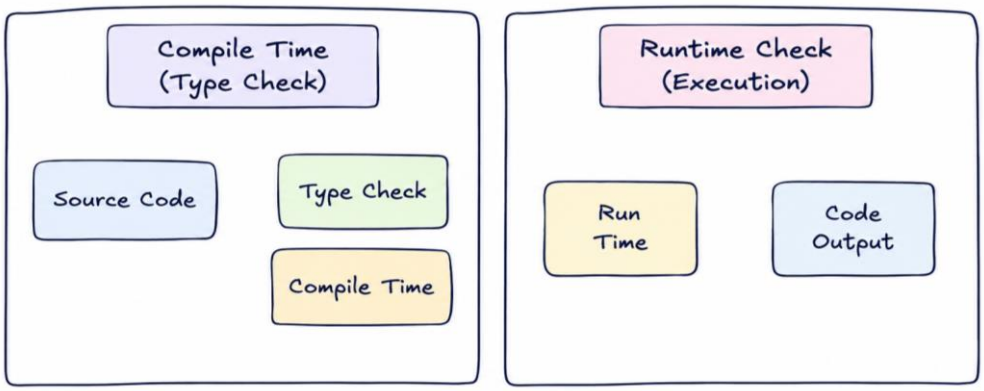


Note:

- In **Client-Side**, JS runs in the browser using the **V8 engine.**
- In **Server-Side**, JS runs using the **Node.js runtime environment** with the **V8 engine.**

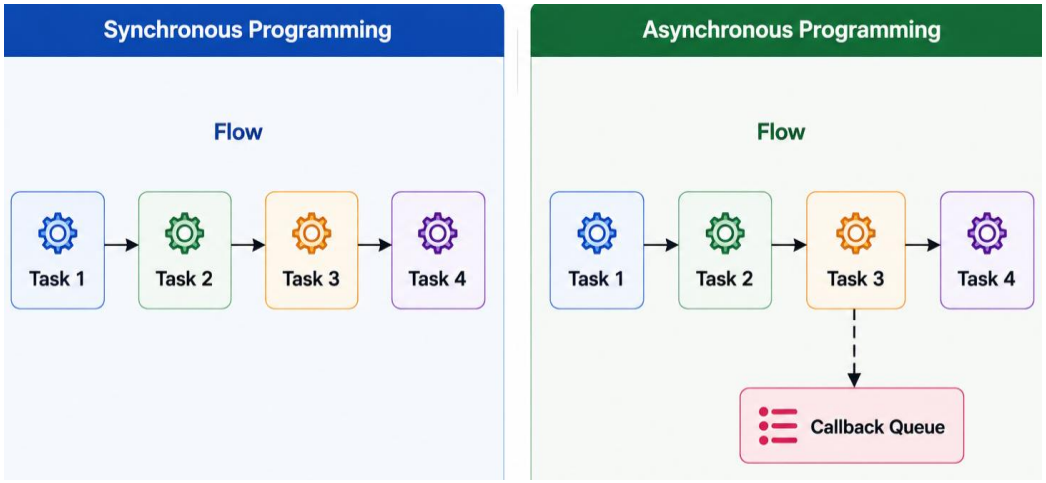
4. Statically Typed vs Dynamically Typed

Statically Typed	Dynamically Typed
Type is checked during compile time.	Type is checked during runtime.
Examples: Java, C++	**Examples:** JavaScript, Python



5. Synchronous vs Asynchronous Programming

Synchronous Programming	Asynchronous Programming
Tasks are executed one after another.	Tasks can run without waiting for another task to finish.
The next task waits until the current task finishes.	Non-blocking execution.
Blocking execution.	Improves performance.

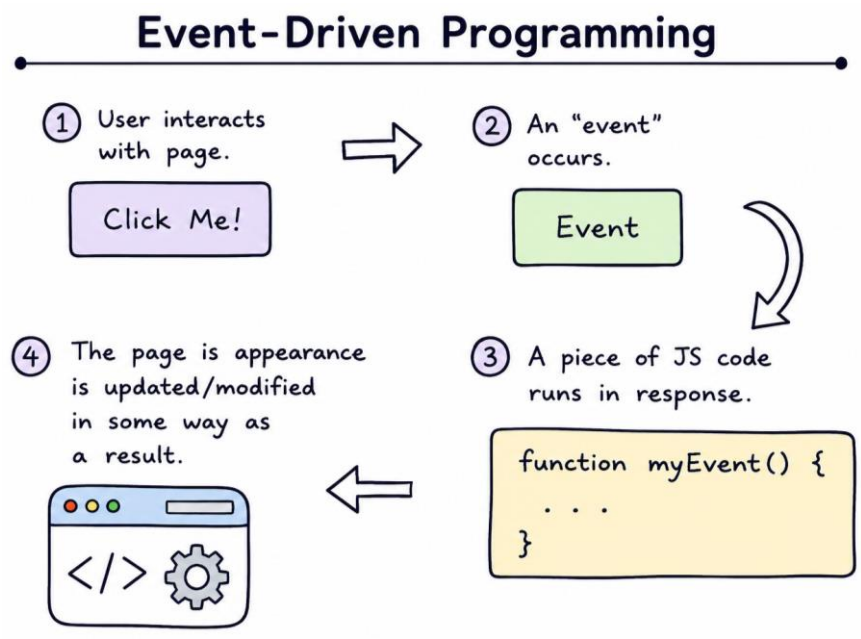


Methods of Asynchronous Programming:

- Callbacks
- Callback Hell
- Promises
- Async / Await

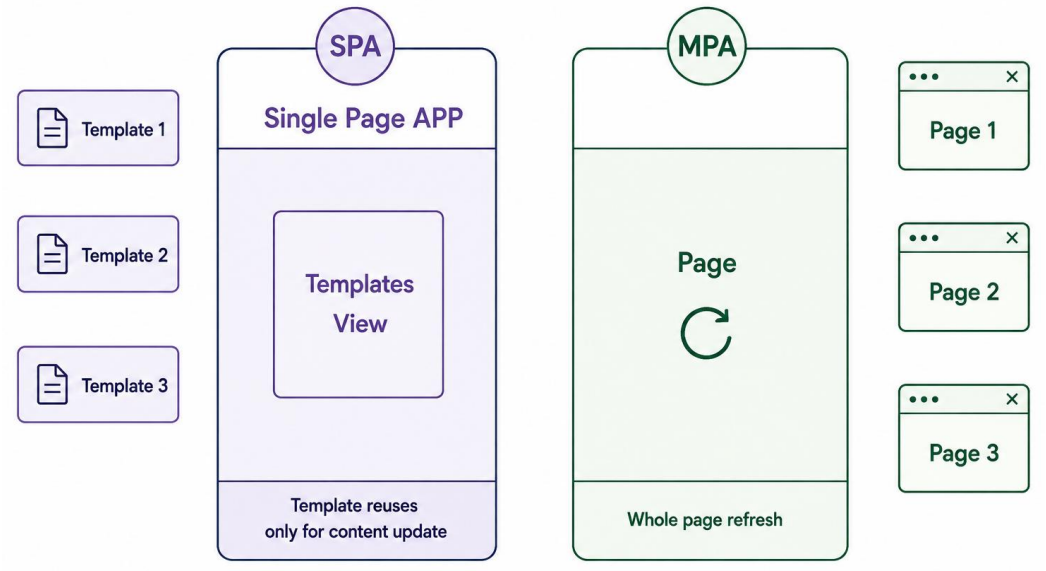
6. Event-Driven Programming

JavaScript responds to user actions or events such as click, keypress, mouseover, input, load, scroll and submit.



7. Single Page Application vs Multi Page Application

Single Page Application (SPA)	Multi Page Application (MPA)
Only the required content is updated.	The entire page reloads on every request.
No full page reload.	Each page is loaded separately from the server.
Provides a faster user experience.	Slower compared to SPA.
Content is dynamically loaded without loading a new HTML page.	Multiple HTML pages are used for different sections.



Examples of SPA frameworks: React, Angular, Vue.js.

8. Object-Based Scripting Language

JavaScript uses objects to organize and manage related data and functionality.

9. Platform Independent

JavaScript can run on different operating systems and devices through a compatible web browser or runtime environment.

10. Why is JavaScript called a High-Level Scripting Language?

JavaScript code is easy to write, read, and understand without dealing directly with low-level system details.

11. What are the Applications of JavaScript?

- **Frontend** – React, Angular, Vue.js
- **Game Development** – Babylon.js, Three.js, Phaser.js
- **Machine Learning** – TensorFlow.js
- **IoT** – Johnny-Five
- **Backend** – Node.js
- **Desktop App Development** – Electron
- **Mobile App Development:**
  - **Native Apps** – React Native
  - **Hybrid Apps** – Ionic, Cordova

Module 3 - JavaScript in HTML

1. How can JavaScript be added to an HTML page?

JavaScript can be added to an HTML page in three ways: Inline JavaScript, Internal JavaScript, and External JavaScript.

2. What is Inline JavaScript?

Inline JavaScript is JavaScript code written directly inside an HTML element using an event attribute.

Example:

```
<button onclick="alert('Hello!')">Click Me</button>
```

3. What is Internal JavaScript?

Internal JavaScript is JavaScript code written inside the <script> tag within the <head> or <body> section of an HTML page.

Example:

```
<script>
   document.write("Welcome to JavaScript!");
</script>
```

4. What is External JavaScript?

External JavaScript is JavaScript code written in a separate .js file and linked to an HTML page using the <script> tag with the src attribute.

Example:

```
<script src="index.js"></script>
```

5. Where should the <script> tag be placed for better performance?

The <script> tag is generally placed at the end of the <body> tag for better performance.

6. What is the use of the src attribute in the <script> tag?

The src attribute is used to specify the path or location of an external JavaScript file linked to an HTML page.

7. What is the difference between Inline, Internal, and External JavaScript?

Method	Where Written	Use	Pros	Cons
Inline JavaScript	Inside HTML element	Small actions (events)	Quick and simple	Not scalable, hard to maintain
Internal JavaScript	Inside <script> in HTML	Single-page scripts	No extra file needed	Not reusable across pages
External JavaScript	Separate .js file (src)	Large projects	Reusable, maintainable, cacheable	Needs extra HTTP request

8. What are the basic syntax rules of JavaScript?

- JavaScript is case-sensitive.
- JavaScript statements can be ended with a semicolon (;).
- Curly braces ({ }) are used to define code blocks.
- Indentation is used to improve code readability.
- Whitespace and line breaks are generally ignored, except inside strings.

Example Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>JavaScript in HTML</title>

    <!-- Internal JavaScript -->
    <script>
        console.log("Hello from Head!");
    </script>
</head>
<body>

    <h1>I'm gonna make you dynamic</h1>

    <!-- Inline JavaScript -->
    <button onclick="alert('Hello JavaScript!')">
        Click Me
    </button>
```

```
    <!-- Internal JavaScript -->
    <script>
        setTimeout(() => {
            document.querySelector("h1").innerHTML = "I'm Changed";
        }, 4000);

        console.log(100);
        console.log("JavaScript");

        document.write("JavaScript INTRO");
    </script>

    <!-- External JavaScript -->
    <script src="index.js"></script>

</body>
</html>
```

Module 4 - Variable Declaration

1. What is a Variable?

Variable is a named container used to store or hold the value in memory, which can be accessed and changed during program execution.

Example:

```
let num = 10;
```

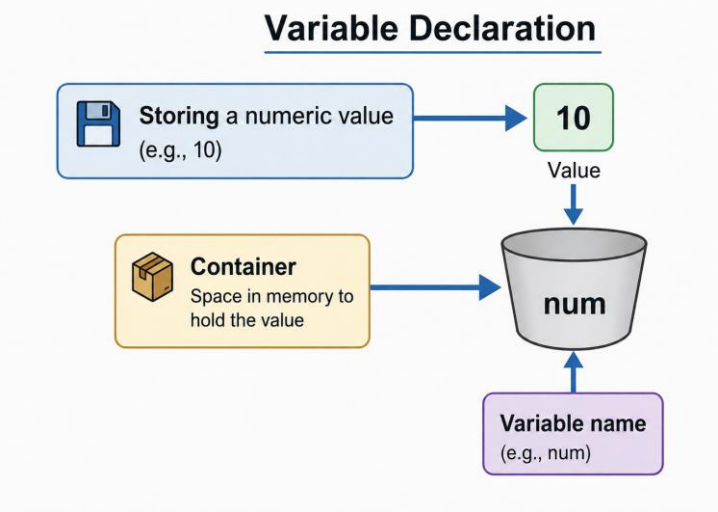
- **let** → Keyword
- **num** → Variable name
- **10** → Value

2. What is Variable Declaration?

Variable Declaration is the process of creating a variable by specifying its name using a keyword such as **var, let, or const.**

Syntax:

```
keyword variable_name = value;
```



3. What are a Variable Name and a Value?

A variable name is the name used to identify a variable, while a value is the actual data stored in that variable.

Example:

```
let age = 20; // age is the variable name and 20 is the value.
```

Example Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Variable Declaration</title>
</head>
<body>

    <script>

        // keyword variable_name = value;

        var userName = "Front End Development";
        let email = "abc@gmail.com";
        const age = 25;

        console.log(userName);
        console.log(email);
        console.log(age);

    </script>

</body>
</html>
```

Module 5 - Keywords and Identifiers

1. What are Keywords?

- Keywords are reserved words with a predefined meaning that cannot be used as identifiers.
- **Example:** var, let, const, if, else, for, while, function, return, class

2. What are Identifiers?

- Identifiers are names given to variables, functions, objects, and other program elements to identify them uniquely.
- **Example**: uName, lName, LName, $lName, _lName

3. What are the rules for naming identifiers?

- Identifiers must have unique names.
- Names must begin with a letter.
- Names cannot begin with a number.
- Names can also begin with $ or _.
- Names are case-sensitive.
- Reserved words cannot be used as identifiers.

Example Code:

```
// Keywords and Identifiers
let uName = "ECMAScript";
console.log(uName);

let lName = "JavaScript";
let LName = "Dynamically Typed";
console.log(lName, LName);

let $lName = "Single Page Application";
let _lName = "SPA";
// let 0lName = "SPA"; // Variable names cannot start with a number
console.log($lName, _lName);
```

4. What are the Identifier Naming Conventions?

Identifier Naming Conventions are rules and guidelines used to name variables, functions, and other identifiers clearly and consistently.

I. Camel Case

→ Starts with lowercase and capitalizes each following word.

→ Commonly used for variable names and function identifiers.

II. Pascal Case

→ Starts with uppercase and capitalizes each following word.

→ Commonly used for objects and constructors.

III. Underscore Case or Snake Case

→ Separates words using underscores (_).

→ Commonly used for variable names, function identifiers, and database fields.

Example Code:

```
// Identifier Naming Conventions
let newemployeeid = 10; // Not Recommended

// 1. Camel Case
let newEmployeeId = 10;

// 2. Pascal Case
let NewEmployeeId = 10;

// 3. Underscore Case or Snake Case
let New_Employee_Id = 10;
let new_Employee_Id = 10;
```

Module 6 - Redeclaration & Reinitialization, Printing Statements

1. What is Declaration?

Declaration is the process of creating a variable without assigning a value.

2. What is Initialization?

Initialization is the process of assigning a value to a variable for the first time.

3. What is Redeclaration?

Redeclaration is the process of declaring the same variable again.

4. What is Reinitialization (Reassigning)?

Reinitialization is the process of assigning a new value to an existing variable.

5. What are the rules for redeclaration and reassignment of var, let, and const variables?

Keyword	Redeclaration	Reassignment
var	✅ Allowed	✅ Allowed
let	❌ Not allowed in the same scope	✅ Allowed
const	❌ Not allowed	❌ Not allowed

6. What are Printing Statements in JavaScript?

Printing Statements are used to display messages, values, or output to the user or browser console.

7. What is the use of alert()?

alert() is used to display a message in a popup box.

8. What is the use of document.write()?

document.write() is used to write content directly to the webpage.

9. What is the use of document.writeln()?

document.writeln() is used to write content to the webpage followed by a line break.

10. What is the use of confirm()?

confirm() is used to display a message with OK and Cancel buttons.

11. What is the use of prompt()?

prompt() is used to display a message and get input from the user.

12. What is the use of console.log()?

console.log() is used to display information in the browser console.

13. What is the use of console.error()?

console.error() is used to display error messages in the browser console.

14. What is the use of console.warn()?

console.warn() is used to display warning messages in the browser console.

15. What is the use of console.clear()?

console.clear() is used to clear the messages displayed in the browser console.

Example Code:

I. Redeclaration & Reinitialization

```
// 1. Var
// var age = 30;   // Declaration & initialization
var age;           // Declaration
age = 40;          // Initialization or Assigning
var age = 80;      // Redeclaration
age = "Eighty";  // Reinitialization or Reassign
console.log(age);

// 2. let
// let newAge = 150;     // Declaration & initialization
let newAge;              // Declaration
newAge = 100;            // Initialization or Assigning
// let newAge = 30;      // Redeclaration is not possible
newAge = "Hundred";  // Reassign
console.log(newAge);

// 3. const
const employeeName = "xyz"; // Declaration & Initialization
// employeeName = "abc";       // Reassign is not possible
console.log(employeeName);
```

II. Printing Statements

```
// 1. alert()
let alertMessage = "Welcome to JavaScript";
alert(alertMessage);

// 2. write()
let writeMessage = "JavaScript";
document.write(writeMessage);

// 3. writeln()
let writelnMessage = "JavaScript";
document.writeln(writelnMessage);

// 4. confirm()
let confirmMessage = "Do you want to continue?";
confirm(confirmMessage);

// 5. prompt()
let promptMessage = "Enter Your Age";
let userInput = prompt(promptMessage);

// 6. console.log()
console.log(userInput);

// 7. console.error()
let errorMessage = "Error";
console.error(errorMessage);

// 8. console.warn()
let warningMessage = "Warning";
console.warn(warningMessage);

// 9. console.clear()
// console.clear();
```

Module 7 - Datatypes and Their Types

1. What is a Datatype?

Datatype is a type of value that defines what kind of data a variable can store.

2. What are Primitive Datatypes?

Primitive Datatypes are basic data types that store a single value, such as Number, String, Boolean, Undefined, and Null.

3. What is a Number?

Number is a datatype used to represent numeric values, including integers and decimal numbers.

Example Code:

```
// Number
var num = 120;
var num = 1.5;
console.log(num); // 1.5
```

4. What is a String?

String is a datatype used to represent text or a sequence of characters, enclosed in single quotes (' '), double quotes (" "), or backticks (` `).

Example Code:

```
// String
let userName = "Javascript is a Scripting Language";
userName = 'Javascript is a Single threaded Language';
userName = `ECMA Script`;
console.log(userName); // ECMA Script
```

5. What is a Boolean?

Boolean is a datatype used to represent two possible values: true or false.

Example Code:

```
// Boolean
let condition = true;
condition = false;
console.log(condition); // false
```

6. What is Undefined?

Undefined is a datatype that represents a variable that has been declared but not assigned a value.

Example Code:

```
// Undefined
// let noValue = undefined;
let noValue;
console.log(noValue); // undefined
```

7. What is Null?

Null is a datatype used to represent an intentional absence of a value.

Example Code:

```
// Null
let emptyValue = null;
console.log(emptyValue); // null
```

8. What are Non-Primitive Datatypes?

Non-Primitive Datatypes (Reference types) are used to store collections of values or complex data, such as arrays and objects.

9. What is an Array?

An array is a data structure used to store multiple values in a single variable. Each value is stored at a specific index, starting from index 0.

Example Code:

```
// Array
// Length                1          2          3          4
let multipleUsers = ["React Js", "Javascript", "Facebook", "Instagram"];
// Index                 0          1          2          3
console.log(multipleUsers);
console.log(multipleUsers[0], multipleUsers[1], multipleUsers[3]);
console.log(multipleUsers.length);
console.log(multipleUsers.length-1);
console.log(multipleUsers[multipleUsers.length-1]);
```

10. What is an Object?

An Object is used to store related information in key-value pairs, where each key represents a property and its value contains the corresponding information.

Example Code:

```
// Object (Key value pair)
let vehicle = {
    vehicleType : "Four Wheeler",
    brand : "Hyundai",
    price : 100000,
    fuelType : "Petrol"
};
console.log(vehicle);
console.log(vehicle.price);
console.log(vehicle.brand);
```

11. What is a Single-Line Comment?

A Single-Line Comment is used to write a comment on one line of code and is written using //.

Example Code:

```
// Single line comment
```

12. What is a Multi-Line Comment?

A Multi-Line Comment is used to write comments across multiple lines of code and is written using /* */.

Example Code:

```
/*
    Multi line comment
    Javascript
    React Js
*/
```

Module 8 - Arithmetic Operators

1. What are Operators?

Operators are symbols or keywords used to perform operations on values and variables, such as arithmetic, relational (comparison), assignment, and logical operations.

2. What are Operands?

- Operands are the values or variables on which an operator performs an operation.
- **Example:** In 10 + 20, 10 and 20 are operands, and + is the operator.

3. What are Arithmetic Operators?

Arithmetic Operators are used to perform mathematical operations on values and variables, such as addition, subtraction, multiplication, division, modulus, exponentiation, increment, and decrement.

Example Code:

```
// Arithmetic Operators
console.log(10 + 20);  // Addition ==> 30
console.log(20 - 5);   // Subtraction ==> 15
console.log(20 * 2);   // Multiplication ==> 40
console.log(20 / 2);   // Division - Quotient ==> 10
console.log(21 % 5);   // Modulus - Remainder ==> 1
console.log(3 ** 4);   // Exponentiation ==> 81

// Increment ( ++ )
let num = 15;
num++; // Post Increment ==> num = num + 1 ==> 15 + 1 = 16
++num; // Pre Increment ==> num = num + 1 ==> 16 + 1 = 17
console.log(num); // 17

// Decrement ( -- )
let num1 = 20;
num1--; // Post Decrement ==> num1 = num1 - 1 ==> 20 - 1 = 19
--num1; // Pre Decrement ==> num1 = num1 - 1 ==> 19 - 1 = 18
console.log(num1); // 18
```

- **Addition (+)** – Used to add two or more values.
- **Subtraction (-)** – Used to subtract one value from another.
- **Multiplication (*)** – Used to multiply two or more values.
- **Division (/)** – Used to divide one value by another and get the quotient.
- **Modulus (%)** – Used to find the remainder after division.
- **Exponentiation (**)** – Used to raise a number to the power of another number.
- **Increment (++)** – Used to increase the value of a variable by 1.
- **Decrement (--)** – Used to decrease the value of a variable by 1.

Module 9 - Post & Pre Increment or Decrement

1. What is Post Increment (x++)?

Post Increment (x++) is used to increase the value of a variable by 1 after using its current value.

Example Code:

```
// Post Increment

// 1. Substitute
// 2. Operation
// 3. Increment

let num = 20;
// let num1 = num++; // Assign current value (20) to num1, then increase num to 21
// console.log(num, num1); // 21 20

let num1 = num++ + num++; // First num++ returns 20, then num becomes 21
                          // Second num++ returns 21, then num becomes 22
                          // num1 = 20 + 21 = 41
console.log(num, num1); // 22 41
```

2. What is Pre Increment (++x)?

Pre Increment (++x) is used to increase the value of a variable by 1 before using its value.

Example Code:

```
// Pre Increment

// 1. Increment
// 2. Substitute
// 3. Operation

let newNum = 40;
// let newNum1 = ++newNum; // First increment newNum to 41, then assign 41 to newNum1
// console.log(newNum, newNum1); // 41 41

let newNum2 = ++newNum + ++newNum; // First ++newNum makes it 41 and returns 41
                                   // Second ++newNum makes it 42 and returns 42
                                   // newNum2 = 41 + 42 = 83
console.log(newNum, newNum2); // 42 83
```

3. What is Post Decrement (x--)?

Post Decrement (x--) is used to decrease the value of a variable by 1 after using its current value.

4. What is Pre Decrement (--x)?

Pre Decrement (--x) is used to decrease the value of a variable by 1 before using its value.

Example Code:

```
// Post Decrement & Pre Decrement

let num2 = 10;

// Post Decrement

// 1. Substitute
// 2. Operation
// 3. Decrement

/*

let num3 = num2-- + --num2; // num2-- returns 10, then num2 becomes 9
                           // --num2 makes num2 8, then returns 8
                           // num3 = 10 + 8 = 18
console.log(num2, num3); // 8 18

*/

// Pre Decrement

// 1. Decrement
// 2. Substitute
// 3. Operation

let num4 = --num2 + num2--; // --num2 makes num2 9 and returns 9
                           // num2-- returns 9, then num2 becomes 8
                           // num4 = 9 + 9 = 18
console.log(num2, num4); // 8 18
```

Module 10 - Assignment & Relational Operators

1. What are Assignment Operators?

Assignment Operators are used to assign or update values in a variable. The basic assignment operator is =.

Example Code:

```
// Assignment Operators
let age = 20;
let additionalVal = 100;

// Adds a value to the variable and assigns the result.
age += 20;                // age = age + 20 ==> 20 + 20 = 40
age += additionalVal; // age = age + additionalVal ==> 40 + 100 = 140

// Subtracts a value from the variable and assigns the result.
age -= 10;                // age = age - 10 ==> 140 - 10 = 130

// Multiplies the variable by a value and assigns the result.
age *= 2;                 // age = age * 2 ==> 130 * 2 = 260

// Divides the variable by a value and assigns the result.
age /= 2;                 // age = age / 2 ==> 260 / 2 = 130

// Assigns the remainder of the division to the variable.
age %= 2;                 // age = age % 2 ==> 130 % 2 = 0

// Raises the variable to the given power and assigns the result.
age **= 2;                // age = age ** 2 ==> 0 ** 2 = 0

console.log(age);
```

2. What are Relational Operators?

Relational Operators, also called Comparison Operators, are used to compare two values and return a Boolean result (true or false).

Example Code:

```
// Relational or Comparison Operators
// Less than
console.log(20 < 20);          // false

// Less than or equal to
console.log(21 <= 20);         // false

// Greater than
console.log(40 > 40);          // false

// Greater than or equal to
console.log(40 >= 39);         // true

// Equal to
console.log(40 == '40');       // true
console.log(40 == 40);         // true

// Not equal to
console.log(40 != '50');       // true
console.log(40 != 50);         // true

// Strict Equal to
// Equal value and same type
console.log(40 === '40');      // false
console.log(40 === 40);        // true

// Strict Not Equal to
// Not equal value and Not same type
console.log(40 !== '40');      // true
console.log(40 !== 40);        // false
```

3. What is the difference between == and ===?

- **== (Loose Equality)** → Compares only value and allows type conversion.
- **=== (Strict Equality)** → Compares value and data type, without type conversion.

Example: 40 == "40" → true, but 40 === "40" → false.

4. What is the difference between != and !==?

- **!= (Loose Inequality)** → Compares only value and allows type conversion.
- **!== (Strict Inequality)** → Compares value and data type without type conversion.

Example: 40 != "40" → false, but 40 !== "40" → true.

Module 11 - Logical Operators

1. What are Logical Operators?

Logical Operators are used to combine two or more conditions and return a Boolean value (true or false).

Example Code:

I. Logical AND (&&)

→ Returns true only when both conditions are true.

```
/*
   Cond1     Cond2     Result
   true      true      true
   true      false     false
   false     true      false
   false     false     false
*/

let age = 18;
console.log(age >= 18 && age <= 30); // true && true = true

let age1 = 35;
console.log(age1 >= 18 && age1 <= 30); // true && false = false

let age2 = 16;
console.log(age2 >= 18 && age2 <= 30); // false && true = false // Short Circuit Evaluation

let idProof = "Aadhaar";
console.log(idProof == "PanCard" && idProof == "Driving License"); // false && false = false
```

Short-Circuit Evaluation

- It stops evaluating conditions as soon as the final result is determined.
- In **&&**, if the first condition is **false**, the remaining conditions are not evaluated.
- In ||, if the first condition is **true**, the remaining conditions are not evaluated.

II. Logical OR (||)

→ Returns true when at least one condition is true.

```
/*
   Cond1     Cond2     Result
   true      true      true
   true      false     true
   false     true      true
   false     false     false
*/

let pass_input = "12345";
console.log(pass_input == "12345" || pass_input == "54321"); // true || false = true

let pass_input1 = "54321";
console.log(pass_input1 == "12345" || pass_input1 == "54321"); // false || true = true

let userName = "abc";
console.log(pass_input1 == "54321" || userName == "abc"); // true || true = true
console.log(pass_input == "98765" || userName == "xyz"); // false || false = false
```

III. Logical NOT (!)

→ Reverses the Boolean value: true becomes false and false becomes true.

```
let num = 10 > 4;
console.log(!num); // false

let num1 = 5 > 20;
console.log(!num1); // true
console.log(!false); // true
console.log(!true); // false
```

Module 12 - Concatenation & Template Strings

1. What is Concatenation?

Concatenation is the process of joining two or more strings together to form a single string using the + operator.

Example Code:

```
// Concatenation
let str = "Hello";
let str1 = "John";
let output = str + str1;
console.log(output);
console.log(str + " " + str1 + " Welcome to our website");
```

2. What are Template Strings?

Template Strings are used to create strings using backticks (`) and allow variables or expressions to be embedded using ${}.

Example Code:

```
// Template Strings or Template Literal ( `` )
let newStr = `Javascript`;

// String Interpolation - to embed variable expression
let newStr1 = `${newStr} is a Scripting Language`;
console.log(newStr1);

// Example
let num = 5;
let first = 1;
console.log(num + " * " + first + " = " + (num*first));
console.log(`${num} * ${first} = ${num*first}`);
```

Module 13 - Type Conversion (Implicit & Explicit)

1. What is Type Conversion?

Type Conversion is the process of converting a value from one data type to another data type.

2. What is Implicit Type Conversion?

Implicit Type Conversion is the automatic conversion of a value from one data type to another by the programming language.

Example Code:

```
// Implicit Type Conversion
let str = "45";
let num = 100;
console.log(str + num);              // 45100
console.log(typeof(str + num));      // string

// String
console.log("Hi" + true);            // Hitrue
console.log("Hi" + undefined);       // Hiundefined
console.log("Hi" + null);            // Hinull
console.log("Hi" + [1,2]);           // Hi1,2
console.log("Hi" + {});              // Hi[object Object]

// String + Anything = String
// Anything + String = String

// Number
console.log(10 + true);              // 10 + 1 = 11
console.log(10 + false);             // 10 + 0 = 10
// true = 1, false = 0
console.log(10 + undefined);         // NaN - Not A Number
console.log(10 + null);              // 10 + 0 = 10
console.log(10 + [1,2]);             // 101, 2
console.log(typeof(10 + [1]));       // string
```

```
console.log(10 + {});              // 10[object Object]
console.log(typeof(10 + ''));      // string
console.log(10 - 'abc');           // NaN
console.log(10 - '');              // 10

// Boolean
console.log(true + 10);            // 11
console.log(true + undefined);     // NaN
console.log(true + null);          // 1
```

3. What is Explicit Type Conversion?

Explicit Type Conversion is the process of manually converting a value from one data type to another using a specific method or function.

Example Code:

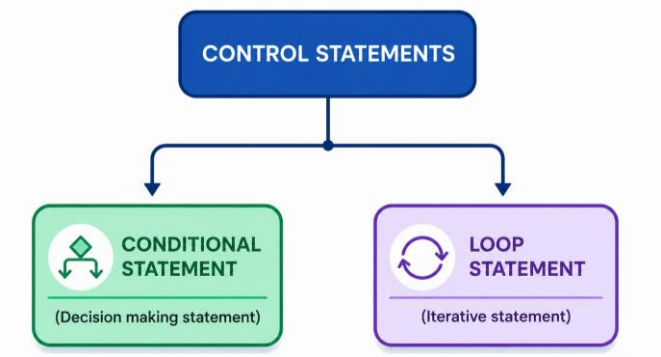
```
// Explicit Type Conversion
// Number
console.log(10 + Number("10"));    // 20
console.log(Number("abc"));        // NaN
console.log(Number(true));         // 1
console.log(Number(false));        // 0
console.log(Number([]));           // 0
console.log(Number([1]));          // 1
console.log(Number([1,2]));        // NaN
console.log(Number({}));           // NaN

// Boolean
console.log(Boolean(""));          // false
console.log(Boolean("123"));       // true
console.log(Boolean(10));          // true
console.log(Boolean(-10));         // true
console.log(Boolean(0));           // false
console.log(Boolean(undefined));   // false
```

```
console.log(Boolean(null));          // false
console.log(Boolean([]));            // true
console.log(Boolean([1]));           // true
console.log(Boolean({}));            // true
console.log(Boolean(Infinity));      // true
console.log(Boolean(-Infinity));     // true
console.log(Boolean(NaN));           // false

// String
console.log(String());               // ""
console.log(String(10));             // "10"
console.log(String(true));           // "true"
console.log(String(false));          // "false"
console.log(String(null));           // "null"
console.log(String(undefined));      // "undefined"
console.log(String([1,2]));          // "1,2"
console.log(String({}));             // "[object Object]"
```

Module 14 - if & if-else Statement

1. What is a control statement?

A control statement is used to control the flow of execution of a program based on given conditions or requirements.



2. What is a conditional statement?

A conditional statement is used to execute a block of code based on whether a given condition is true or false.

3. What is a looping statement?

A looping statement is used to repeatedly execute a block of code as long as a specified condition is satisfied.

4. What is an if Statement?

An if statement is used to check a condition and execute a block of code if the condition is true.

Example Code:

```
// if Statement
/*
   if(condition){
      Statement
   }
*/

let uAge = 25;
if(uAge >= 18){
   console.log("He is eligible to vote");
}
```

5. What is an if-else statement?

An if-else statement is used to check a condition and execute one block if the condition is true, otherwise it executes the else block.

Example Code:

```
// if-else Statement
/*
   if(condition){
      Statement
   }
   else{
      Statement
   }
*/

// Check voting eligibility
let uAge1 = 17;
if(uAge1 >= 18){
   console.log("He is eligible to vote");
}
else{
   console.log("He is not eligible to vote");
}

// Check password input
let pass_input = true;
if(pass_input){
   console.log("Welcome to Website");
}
else{
   console.log("Password is Incorrect");

}
```

Module 15 - if, else-if, else Statement

1. What is an if, else-if, else Statement?

An if, else-if, else statement is used to check multiple conditions one by one and executes the block for the first matching condition; if no condition matches, the else block executes.

Example Code:

```
// if, else-if, else Statement
/*
   if(condition 1){
      Statement
   }
   else if(condition 2){
      Statement
   }
   else if(condition 3){
      Statement
   }
   else{
      Statement
   }
*/

// Check the hour and display the time period
let hour = 12;
// else-if ladder
if(hour >= 1 && hour <= 6){
   console.log("Early Morning");
}
else if(hour >= 7 && hour <= 12){
   console.log("Morning");
}
else if(hour >= 13 && hour <= 18){
   console.log("Noon");
}
```

```
else if(hour >= 19 && hour <= 24){
    console.log("Night");
}
else{
    console.log("It is not an valid hour");
}

// Check the mark and display the grade
let mark = 85;
// else-if ladder
if(mark >= 41 && mark <= 60){
    console.log("E-Grade");
}
else if(mark >= 61 && mark <= 80){
    console.log("C-Grade");
}
else if(mark >= 81 && mark <= 90){
    console.log("A-Grade");
}
else if(mark >= 91 && mark <= 100){
    console.log("S-Grade");
}
else{
    console.log("Arrear");
}
```

Module 16 - switch Statement

1. What is a Switch Statement?

A switch statement is used to choose one option from multiple choices based on the value of a variable or expression.

Example Code:

```
// Switch Statement
/*
   switch(expression){
      case value: Statement;
      break;
      case value: Statement;
      break;
      default: Statement
      break;
   }
*/

let trafficLight = "Over Speed";
switch (trafficLight) {
   case "red":
      console.log("Stop");
      break;
   case "yellow":
      console.log("Slow Down");
      break;
   case "green":
      console.log("Go");
      break;
   default:
      console.log("Pay Fine");
      break;
}
```

2. What is the Fall-Through Method?

The Fall Through Method is a technique in a switch statement where multiple cases are grouped together without break, so they execute the same block of code.

Example Code:

```
// Fall Through Method
let day = "Saturday";
switch(day){
    case "Monday":
    case "Tuesday":
    case "Wednesday":
    case "Thursday":
    case "Friday":
        console.log("WeekDay");
        break;
    case "Saturday":
    case "Sunday":
        console.log("WeekEnd");
        break;
    default:
        console.log("It is not a day");
        break;
}
```

Module 17 - Nested if Statement & Ternary Operator

1. What is a Nested if Statement?

A nested if statement is an if statement placed inside another if statement. It is used to check multiple conditions one after another.

Example Code:

```
// Nested if Statement
let age = 25;
let height = 160;
let waight = 60;
if(age >= 18){
    if(height >= 150){
        if(waight >= 45){
            console.log("You are Selected");
        }
        else{
            console.log("Waight is not Matched");
        }
    }
    else{
        console.log("Height is not matched");
    }
}
else{
    console.log("Age is not valid");
}
```

2. What is a Ternary Operator?

A ternary operator is a short way of writing an if-else statement. It checks a condition and returns one of two values.

Example Code:

```
// Ternary Operator
/*
   condition ? "Statement1" : "Statement2"
*/


let pass_input = true;
pass_input ? console.log("Welcome to website") : console.log("Password Incorrect");
```

Module 18 - for Loop

1. What is a for loop?

A for loop is a loop used to repeat a block of code a specific number of times based on a condition.

Example Code:

```
// for Loop
/*
    for(initialization; condition; counter){
    }
*/

// for(let i = 1; i <= 10; i++){
//     console.log(i);
// }

for(let i = 1; i <= 10; i++){
    // if(i % 2 === 0) console.log(i);
    if(i % 2 !== 0) console.log(i);
}
```

Module 19 - while Loop

1. What is a while loop?

A while loop is a loop that repeatedly executes a block of code as long as a given condition is true.

Example Code:

```
// while Loop
/*
   initialization;
   while(condition){
      Statement;
      counter;
   }
*/

// let val = 10;
// while(val >= 1){
//     console.log(val);
//     val--;
// }

let num = 1234;
let sum = 0;
// 1 + 2 + 3 + 4 = 10
while(num > 0){
   let last = num % 10;
   num = parseInt(num / 10);
   sum = sum + last;
}
console.log(sum);
```

Module 20 - do-while Loop, break & continue

1. What is a do-while loop?

A do-while loop is a loop that executes the code at least once, and then checks the condition.

Example Code:

```
// do-while Loop
let num = 21;
do{
   console.log(num);
   num++;
}
while(num <= 20);
```

2. What is a break statement?

The break statement is used to immediately stop a loop or switch statement, even if the loop condition is still true.

Example Code:

```
// break
for(let i=1; i <= 10; i++){
   if(i % 2 == 0){
      if(i === 10){
         break;
      }
      console.log(i);
   }
}
```

3. What is a continue statement?

The continue statement is used to skip the current iteration of a loop and move to the next iteration.

Example Code:

```
// continue
for(let i=1; i <= 20; i++){
    if(i === 10){
        continue;
    }
    console.log(i);
}
```

Module 21 - JavaScript Functions

1. What are JavaScript Functions?

JavaScript Functions are reusable blocks of code designed to perform a specific task when they are called.

Example Code:

```
let person1 = {
    name1: "John",
    age: 30
};
// Function definition
function printUserName(uName, uAge){
    // Checks whether age is less than 40.
    if(uAge < 40){
        // Prints the username and age.
        console.log(`Hi ${uName}, your age is ${uAge}`);
    }
}
// Calling function with different arguments.
printUserName("Kesavan", 40);
printUserName("Murugesan", 60);
printUserName("David", 25);
// Passing object properties as function arguments.
printUserName(person1.name1, person1.age);
```

Module 22 - Default Parameters & Default Values

1. What are Default Parameters?

Default Parameters are values assigned to function parameters that are used automatically when no value or undefined is passed.

Example Code:

```
// Function with default parameters
function printUserName(uName = "Johny", uAge = 30){
    console.log(`Hi ${uName}, your age is ${uAge}`);
}
// Pass values to the parameters.
printUserName("Kesavan", 40);
printUserName("Murugesan", 60);
printUserName("David", 25);
// undefined uses the default age.
printUserName("Madhan", undefined);
// undefined uses the default name.
printUserName(undefined, 35);
// No arguments uses both default values.
printUserName();
```

2. What are Default Values?

Default Values are fallback values assigned to a variable or expression when the original value is missing or falsy.

Example Code:

```
// let employeeId = "IFS12345";
let employeeId = "";
// Use "UQI123" if employeeId is falsy.
let newId = employeeId || "UQI123";
console.log(newId);
```

Module 23 - Return & Non-return Types

1. What is Return Type Function?

A return type is a function that uses the **return** statement to send a value back to the place where the function was called.

Example Code:

```
function findRectArea(l, b){
    // Returns the calculated value.
    let condition = true;
    if(condition){
        return l * b;
    }
    else{
        return null;
    }
}
// Stores the returned value.
let newVal = findRectArea(20, 10);
console.log(findRectArea(100, 50), newVal);

function cubic(num){
    // Returns the calculated cube value.
    return num ** 3;
}
let newVal1 = cubic(4);
console.log(newVal1);
```

2. What is Non-return Type Function?

A non-return type is a function that does not return a value. If there is no **return** statement, JavaScript returns **undefined**.

Example Code:

```
function name(){
    // Only prints a message.
    console.log("Non-return type");
}
// name();
// Function returns undefined because there is no return statement.
let noReturn = name();
console.log(noReturn);
```

Module 24 - Variable Scope

1. What is Function Scope?

Function Scope means a variable declared inside a function can be accessed only within that function.

Example Code:

```
function outerFunction(){
    // var is accessible throughout the function.
    if(true){
        var functionVar = "Im a Variable";
    }
    console.log(functionVar);
}
outerFunction();
```

2. What is Block Scope?

Block Scope means a variable declared inside a block { } can be accessed only within that block.

Example Code:

```
function blockScoped(){
    // Accessible inside this function block.
    let blockVariable = "Im a Block scoped variable";
    if(true){
        // Accessible only inside this if block.
        const blockVariable1 = "Im a const variable";
        console.log(blockVariable);
        console.log(blockVariable1);
    }
}
blockScoped();
```

3. What is Global Scope?

Global Scope means a variable declared outside all functions and blocks can be accessed from anywhere in the program.

Example Code:

```
var a = 10;
let b = 20;
const c = 30;
function accessGlobalLocalVar(){
   // Local variables can be declared here.
   // var a = 101;
   // let b = 202;
   // const c = 303;
   function innerFunction(){
      // Inner function can access variables from outer/global scope.
      // var a = 100;
      // let b = 200;
      // const c = 300;
      console.log(a + b + c);
   }
   innerFunction();
   console.log(a + b + c);
}
accessGlobalLocalVar();
console.log(a + b + c);
```

- **var** → Function scoped, can redeclare and reassign.
- **let** → Block scoped, can reassign but cannot redeclare.
- **const** → Block scoped, cannot reassign or redeclare.

Module 25 - Function Types

1. What is a Named Function?

A Named Function is a function that has a specific name and can be called using that name.

Example Code:

```
// Named Function
function add(uName){
    console.log("Named Function " + uName);
}
add("Im a Function");
```

2. What is an Anonymous Function?

An anonymous function is a function that does not have a name and is commonly used as a callback or assigned to a variable.

Example Code:

```
// Anonymous Function
let AnonFun = function(){
    console.log("Anonymous Function");
}
AnonFun();
```

3. What is an Arrow Function?

An Arrow Function is a shorter way to write a JavaScript function using the => operator. It was introduced in ES6 (ECMAScript 2015).

Example Code:

```
// Arrow Function
let arrowFun = () => {
    console.log("Arrow Function");
}
arrowFun();
```

4. What is a Higher-Order Function?

A Higher Order Function is a function that accepts another function as an argument or returns a function.

5. What is a Callback Function?

A Callback Function is a function passed as an argument to another function and executed inside that function.

Example Code:

```
// Higher Order Function And Call Back Function
function function1(callback){
    console.log("Im a Higher Order Function");
    callback();
}
function function2(){
    console.log("Im a Call Back Function");
}
function1(function2);
```

Module 26 - Hoisting (Variables & Functions)

1. What is Hoisting?

Hoisting is a JavaScript mechanism where variable and function declarations are moved to the top of their scope during the compilation phase.

2. Variable Hoisting

- **var** → Declaration is hoisted and initialized with **undefined**.
- **let / const** → Declaration is hoisted but stays in the **Temporal Dead Zone (TDZ)** until initialization.

3. Function Hoisting

- Function declarations are fully hoisted.
- They can be called before their declaration in the code.

4. Function Expression Hoisting

- A function stored in a variable is not fully hoisted.
- With **var** → the variable is undefined before assignment, so calling it causes a **TypeError**.
- With **let/const** → accessing it before initialization causes a **ReferenceError**.

Example Code:

```
// Variable Declaration Hoisting

/* Before Code Execution - Declaration will come at top
    var a       → declaration is hoisted
    let b       → declaration is hoisted but stays in TDZ
    const c     → declaration is hoisted but stays in TDZ
   sample()     → function declaration is fully hoisted
   sample1      → declaration depends on var/let
*/

// var is hoisted with the value undefined
console.log(a);
var a = 10;
console.log(a);

// let is in the Temporal Dead Zone (TDZ)
console.log(b); // ReferenceError
let b = 20;
console.log(b);
```

```
// const is in the Temporal Dead Zone (TDZ)
console.log(c); // ReferenceError
const c = 30;
console.log(c);


// Function Declaration Hoisting
// Function declaration can be called before its definition.
sample();
function sample(){
    console.log("IM in");
}


// Function Expression Hoisting
// let sample1
// var sample1
console.log(sample1);
sample1();
// var function expression gives TypeError when called before assignment.
// var sample1 = function(){
//     console.log("Am i accessible");
// }
// let function expression gives ReferenceError before initialization.
let sample1 = function(){
    console.log("Am i accessible");
}
sample1();
```

Hoisting Comparison Table

Feature	var	let	const	Function Declaration	Function Expression
Hoisted ?	✅ Yes	✅ Yes	✅ Yes	✅ Yes (Fully)	❌ No
Initialized ?	✅ Yes (as undefined)	❌ No	❌ No	❌ No (until definition)	❌ No
Accessible Before Declaration ?	✅ Yes	❌ No	❌ No	✅ Yes	❌ No
Error Before Declaration ?	❌ No	✅ Yes (ReferenceError)	✅ Yes (ReferenceError)	❌ No	✅ Yes (TypeError when called)
TDZ (Temporal Dead Zone)	❌ No	✅ Yes	✅ Yes	❌ No	N/A

Module 27 – Currying

1. What is Currying?

Currying is a technique of converting a function that takes multiple arguments into a sequence of functions, where each function takes one argument at a time.

Example Code:

```
// UnCurrying Function
// function add(a, b, c){
//     console.log(a + b + c);
// }
// add(1, 2, 3);

// Currying Function
// First function takes the first value.
function add(a){
    // Second function takes the second value.
    return function(b){
        // Third function takes the third value.
        return function(c){
            // Add all three values.
            console.log(a + b + c);
        }
    }
}
// Pass values one by one.
add(10)(20)(30);
// Store the first function.
let curry1 = add(100);
// Store the second function.
let curry2 = curry1(200);
// Pass the final value.
curry2(300);
// Check returned functions.
// console.log(curry1);
// console.log(curry2);
```

Module 28 - Self-invoked Functions & Closures

1. What is a Self-invoked Function (IIFE)?

A Self-invoked Function, also called an IIFE (Immediately Invoked Function Expression), is a function that executes immediately after it is created, without calling it separately.

Example Code:

```
// Executes immediately
(function (userName, age) {
    console.log("Self-invoked Function", userName + age);
})("Deveeswar", 23);
```

2. What is Closure?

A closure is a function that remembers and can access variables from its outer function even after the outer function has finished executing.

Example Code:

```
 // Outer function
function outerFunction() {
    // Outer variable
    let outerVariable = "I'm from outer scope";
    // Inner function
    function innerFunction() {
        console.log(outerVariable);
    }
    // Return inner function
    return innerFunction;
}
// Store returned function
let innerFun = outerFunction();
// Call inner function
innerFun();
```

Module 29 - Generator Functions

1. What is a Generator Function?

A Generator Function is a special type of function that can pause its execution using the yield keyword and continue from the same point when next() is called. It is declared using function*.

Example Code 1:

```
// Generator Function
// 'function*' declares a generator function.
function* generatorFunction() {
    // Pauses the function and returns "First Val".
    yield "First Val";
    // Continues from the previous pause and returns "Second Val".
    yield "Second Val";
    // Continues again and returns "Third Val".
    yield "Third Val";
    // Ends the generator and returns the final value.
    return "Final Val";
}
// Creates a generator object.
let generator = generatorFunction();
// 'next()' resumes the function until the next 'yield' or 'return'.
console.log(generator.next().value);
console.log("Im executing after first yield statement");
console.log(generator.next().value);
console.log(generator.next().value);
console.log(generator.next().value);
```

Example Code 2:

```
function* url() {
    // Returns each part of the URL one by one.
    yield "https:/";
    yield "www.uniqtech.com";
    yield "homePage";
}
// Creates a generator object.
let origin1 = url();
// Each 'next()' returns the next yielded value.
console.log(origin1.next().value);
console.log(origin1.next().value);
console.log(origin1.next().value);
// No more values are available, so it returns undefined.
console.log(origin1.next().value);
```

Module 30 - Dense vs Sparse Arrays

1. What is Array?

An array is a data structure used to store multiple values in a single variable. Each value is stored at a specific index, starting from index 0.

Example Code:

```
// Array
let flavors = ["Vanilla", "Butterscortch", "Lavendar", "Chocolate"];
// Accesses array elements using index numbers.
console.log(flavors);
console.log(flavors[0]);
console.log(flavors[1]);
console.log(flavors[2]);
console.log(flavors[3]);
// Returns the last element.
console.log(flavors[flavors.length - 1]);
```

2. Rules & Features of Arrays

- An array is a data structure used to store multiple values in a single variable.
- An array can store different data types (numbers, strings, booleans, objects, etc.).
- An array can store homogeneous (same data type) or heterogeneous (different data types) values.
- Array elements are accessed using their index, and indexing starts from 0.
- Arrays are dynamic (flexible) in size, so they can grow or shrink when elements are added or removed.

3. Types of Array Creation

- Array Literal
- Array Constructor

4. What is Array Literal?

Array Literal is the simplest and most commonly used way to create an array using square brackets [].

Example Code:

```
// Array Literal
let sample = [1, "two", true, null, undefined, { id: 1 }];
console.log(sample);
```

5. What is Array Constructor?

Array Constructor is a way to create an array using the new Array() constructor.

Example Code:

```
// Array Constructor
let newArray = new Array();
// Adds values to the array using index numbers.
newArray[0] = "First";
newArray[1] = "Second";
newArray[2] = "Third";
newArray[3] = "Fourth";
console.log(newArray);
// Returns the total number of elements in the array.
console.log(newArray.length);
```

6. Dense vs Sparse Arrays

- **Dense Array**: An array in which every index contains a value, with no empty spaces.
- **Sparse Array**: An array in which one or more indexes are empty (missing values).

Example Code:

```
// Dense Array
let denseArray = [1, 2, 3, 4, 5]; // Uses contiguous memory locations.
// Example Base Addresses: 1004, 1008, 1012, 1016, 1020
// Memory Address = Base Address + (Index * Size)
//                = 1004 + (0 * 4) = 1004
console.log(denseArray);


// Sparse Array
let sparseArray = [10, 20, , 40, , 60]; // Uses a Hash Table or Hash Map internally.
console.log(sparseArray);
```

Module 31 - Object Shorthand Assigned Properties

1. What is Object Shorthand Assigned Properties?

Object Shorthand Assigned Properties is a JavaScript feature that allows you to create object properties using variable names directly when the property name and variable name are the same.

2. What is Dynamic Property?

A Dynamic Property is a JavaScript feature that allows you to create or access an object property using a variable or expression inside square brackets [].

Example Code:

I. Object & Methods

```
let userProfile = {
    userName: "Kesavan",
    age: 25,
    hairColor: "Black",
    eyeColor: "Brown",
    // Method inside the object.
    eat: function() {
        console.log("Im gonna eat ice cream");
        return "Vanilla Ice Cream";
    }
};
console.log(userProfile.userName);
console.log(userProfile.age);
console.log(userProfile.hairColor);
// Calls the object method and stores its return value.
let iceCreamType = userProfile.eat();
console.log(iceCreamType);
```

II. Object Property Access

```
let vehicle = {
   "vehicleType": "four-wheeler",
   "price": 20000,
   fuelType: "petrol",
   "seater type": ["two", "three", "four"]
};
console.log(vehicle.vehicleType);
// Bracket notation is used to access properties with quotes or spaces.
console.log(vehicle["price"]);
console.log(vehicle["fuelType"]);
console.log(vehicle["seater type"]);
console.log(vehicle["seater type"][1]);
```

III. Object Shorthand Assigned Properties & Dynamic Property

```
let uName = "Kesavan";
let uAge = 30;
let dynamicProp = "employeeId";
let person1 = {
   // Shorthand Property: Property name and variable name are the same.
   uName,
   uAge,
   // Dynamic Property: Creates a property with the name "dynamicProp".
   ["dynamicProp"]: "IFS246",
   // Dynamic Property: Uses the value of the variable (employeeId) as the property name.
   [dynamicProp]: "UQI!@#$",
};
console.log(person1);
```

```
// Accesses shorthand properties using dot notation.
// Accesses dynamic properties using dot notation and bracket notation.
console.log(
    person1.uName,
    person1.uAge,
    person1.dynamicProp,
    person1[dynamicProp]
);
```

Module 32 - Iterate over Array & String

1. What is Iteration?

Iteration is the process of accessing each element of an array or each character of a string one by one using a loop.

Example Code:

I. Iterating Over an Array

```
let arr = [10, 20, 30, 40];
// 'length' stores the total number of elements in the array.
let length = arr.length;
// The loop runs from index 0 to the last index of the array.
for (let i = 0; i < length; i++) {
    // Accesses each array element using its index.
    console.log(arr[i]);
}
```

II. Iterating Over a String

```
let str = "Javascript";
// The loop runs from index 0 to the last character of the string.
for (let i = 0; i < str.length; i++) {
    // Accesses each character using its index.
    console.log(str[i]);
}
```

Module 33 - for…of Loop with Generator Function

1. What is for...of Loop?

The for...of loop is used to iterate over iterable objects such as arrays, strings, and generator objects. It returns one value at a time.

Example Code:

I. for...of Loop with Array

```
let arr = [10, 100, 1000, 10000];
// 'for...of' accesses each array element one by one.
for (let val of arr) {
    console.log(val);
}
```

II. for...of Loop with String

```
let str = "Javascript";
// 'for...of' accesses each character of the string one by one.
for (let char of str) {
    console.log(char);
}
```

III. for...of Loop with Generator Function

```
// Generator function that yields one value at a time.
function* genFunction() {
    yield "One";
    yield "Two";
    yield "Three";
}
// Creates a generator object.
let iterator = genFunction();
// 'for...of' automatically gets each yielded value until the generator ends.
for (let val of iterator) {
    console.log(val);
}
```

Module 34 - for…in Loop (Object, Array & String)

1. What is for...in Loop?

The for...in loop is used to iterate over the keys (property names or indexes) of an object, array, or string.

Example Code:

I. for...in Loop with Object

```
let person1 = {
    uName: "Kesavan",
    hobbies: ["Cricket", "Video Maker", "Editor"],
    familyDetails: {
        totalMembers: 5,
        siblings: ["a", "b", "c"]
    },
    walk() {
        console.log("Im going to home");
    }
};
// 'key' stores each property name of the object.
for (let key in person1) {
    // Accesses the value of each property using the key.
    console.log(person1[key]);
}
```

II. for...in Loop with Array

```
let arr = [120, 130, 140, 150];
// 'key' stores the index of each array element.
for (let key in arr) {
    // Accesses each element using its index.
    console.log(arr[key]);
}
```

III.for...in Loop with String

```
let str = "ECMA Script";
// 'key' stores the index of each character.
for (let key in str) {
    // Accesses each character using its index.
    console.log(str[key] + 1);
}
```

Module 35 - Spread Operator & Rest Parameter

1. What is Spread Operator?

The Spread Operator (...) is a JavaScript operator used to expand or copy the elements of an array or the properties of an object into another array or object.

Example Code:

I. Spread Operator (Array)

```
let hobbies = ["Cricket", "FootBall", "BasketBall"];
let hobbies1 = ["Reader", "Writer"];
// Combines both arrays into a new array.
let newArray = [...hobbies, ...hobbies1];
// Adds new elements to the copied array.
let newArray1 = [...newArray, "VideoCreator", "Content Writer"];
console.log(newArray);
console.log(newArray1);
```

II. Spread Operator (Object)

```
let empDetail = {
   empId: "IQ123",
   empName: "Kesavan",
   empRole: "React Developer"
};
// Copies all properties and updates/adds new properties.
let team2 = {
   ...empDetail,
   empId: "IQ456",
   empSalary: 100000,
   team2Desig: "Full Stack Developer"
};
console.log(empDetail);
console.log(team2);
```

2. What is Rest Parameter?

The Rest Parameter (...) is a JavaScript feature used in function parameters to collect multiple arguments into a single array. It must always be the last parameter.

Example Code:

```
// '...arr' collects all remaining arguments into an array.
function restParams(a, b, ...arr) {
    console.log(a, b, arr);
}
restParams(1, 2, 3, 4, 5);
```

Module 36 - Destructuring & Nested Destructuring

1. What is Destructuring?

Destructuring is a JavaScript feature that allows us to extract values from an array or properties from an object and store them in variables easily.

2. What is Nested Destructuring?

Nested Destructuring is a JavaScript feature that allows us to extract values from nested arrays or nested objects and store them in variables easily.

Example Code:

I. Array Destructuring

```
let arr = [10, 20, 30, 40];
// Destructuring: Stores the 1st value in 'a' and the 4th value in 'd'.
// Empty commas (,) skip the unwanted values.
let [a, , , d] = arr;
console.log(a, d);
let arr1 = [10, 20, 30, 40, 5, 4, 5, 6, 7, 8, 9, 10];
// '...a4' (Rest Operator) stores all remaining values in an array.
let [a1, a2, a3, ...a4] = arr1;
console.log(a1, a2, a3, a4);
```

II. Array Nested Destructuring

```
let nestArr = [1, 2, 3, 4, [10, 20, [30, 40]]];
// Extracts values from the nested arrays directly into variables.
let [A, B, C, D, [A1, A2, [B1, B2]]] = nestArr;
console.log(A, B, C, D, A1, A2, B1, B2);
```

III. Object Destructuring

```
let student = {
    name: "Rahul",
    age: 22,
    course: "MCA",
    city: "Puducherry"
};
// Extracts only the 'name' and 'city' properties from the object.
let { name, city } = student;
console.log(name, city);
let employee = {
    empId: 101,
    empName: "John",
    department: "IT",
    salary: 50000,
    location: "Chennai"
};
// '...empDetails' stores the remaining properties in a new object.
let { empId, empName, ...empDetails } = employee;
console.log(empId, empName, empDetails);
```

IV. Object Nested Destructuring

```
let person = {
    id: 1,
    name1: "Devesh",
    address: {
        city1: "Puducherry",
        state: "Tamil Nadu",
        pin: {
            code: 605110,
            area: "Villianur"
        }
    }
};
// Extracts values directly from nested objects into variables.
let {
    id,
    name1,
    address: {
        city1,
        state,
        pin: { code, area }
    }
} = person;
console.log(id, name1, city1, state, code, area);
```

Module 37 - Array Methods (splice, pop, push, shift, unshift)

1. What are Array Methods?

Array methods are built-in JavaScript functions used to perform different operations on arrays, such as adding, removing, searching, modifying, sorting, and processing elements.

2. pop() Method

The pop() method is used to remove the last element from an array.

Example Code:

```
let arr = [100, 200, 300, 400];
// Removes the last element from the array and returns it.
let poppedVal = arr.pop();
console.log(poppedVal, arr);
```

3. push() Method

The push() method is used to add one or more elements to the end of an array.

Example Code:

```
// Adds one or more elements to the end of the array.
arr.push(500, 550, 600);
console.log(arr);
```

4. shift() Method

The shift() method is used to remove the first element from an array.

Example Code:

```
// Removes the first element from the array and returns it.
let shiftedVal = arr.shift();
console.log(shiftedVal, arr);
```

5. unshift() Method

The unshift() method is used to add one or more elements to the beginning of an array.

Example Code:

```
// Adds one or more elements to the beginning of the array.
arr.unshift(50, 75);
console.log(arr);
```

6. splice() Method

The splice() method is used to add, remove, or replace elements at a specific index in an array.

Example Code:

```
// Syntax: array.splice(startIndex, deleteCount, item1, item2, ...)

// Delete using splice
// Removes 2 elements starting from index 2 and returns them.
let removedItems = arr.splice(2, 2);
console.log(removedItems, arr);

// Insert using splice
// Inserts 250 and 350 at index 2 without removing any elements.
arr.splice(2, 0, 250, 350);
console.log(arr);

// Replace using splice
// Replaces 2 elements starting from index 4 with 800 and 900.
arr.splice(4, 2, 800, 900);
console.log(arr);
```

Module 38 - Array Methods (concat, slice, flat & fill)

1. concat() Method

The concat() method is used to combine two or more arrays and returns a new array without changing the original arrays.

Example Code:

```
let arr = [1, 2, 3, 4];
let arr1 = [4, 5, 6, 7];
// Combines two arrays and returns a new array.
let newArr = arr.concat(arr1);
// Adds values to the end and returns a new array.
let newArr1 = arr.concat(10, 20, 30);
// Creates a shallow copy of the array.
let newArr2 = [].concat(arr);
arr[0] = 111;
console.log(newArr, newArr1, newArr2, arr);
```

2. slice() Method

The slice() method is used to extract a portion of an array and returns a new array without changing the original array.

Example Code:

```
let newArray = [10, 2, 3, 4, 5, 6, 7];
// Creates a copy of the entire array.
let slicedVal = newArray.slice();
newArray[0] = 101;
// Extracts elements from index 1 to the end.
let slicedVal1 = newArray.slice(1);
// Extracts elements from index 1 to index 3.
let slicedVal2 = newArray.slice(1, 4); // (start, end - 1) (1, 4 - 1 = 3)
// Extracts elements from index 0 to index 2.
let slicedVal3 = newArray.slice(0, 3); // (start, end - 1) (0, 3 - 1 = 2)
console.log(slicedVal, slicedVal1, slicedVal2, slicedVal3);
```

3. flat() Method

flat() Method is used to convert nested arrays into a single-level array by removing the specified level of nesting.

Example Code:

```
let Arr = [1, 2, 3, [4, [5, 6, [40, 50, [70, 80]]]]];
// Flattens the array up to depth 2.
let NewArr = Arr.flat(2);
// Flattens all nested arrays into a single array.
let NewArr1 = Arr.flat(Infinity);
console.log(Arr, NewArr, NewArr1);
```

4. fill() Method

The fill() method is used to replace elements of an array with a specified value from a given start index to an end index.

Example Code:

```
let ARR = [10, 20, 30];
// Fills the array with 101 from index 0 to index 1.
ARR.fill(101, 0, 2); // (value, start, end - 1) (101, 0, 2 - 1 = 1)
// Fills the array with 103 at index 2.
ARR.fill(103, 2, 3); // (value, start, end - 1) (103, 2, 3 - 1 = 2)
console.log(ARR);
```

Module 39 - Array Methods (sort, reverse, includes, join & toString)

1. sort() Method

The sort() method is used to sort the elements of an array. By default, it sorts elements in ascending ASCII (Unicode) order.

Example Code:

```
let arr = [5, 1, 4, 6, 2, 8, 10, 20, 15, 45, 101, 111, 26, 345, "&", " "];
// Sorts the array in ASCII (Unicode) order.
arr.sort();
console.log(arr);
```

2. reverse() Method

The reverse() method is used to reverse the order of the elements in an array.

Example Code:

```
let arr1 = [10, 20, 30, 40];
// Reverses the order of the array elements.
arr1.reverse();
console.log(arr1);
```

3. includes() Method

The includes() method is used to check whether an array contains a specified element. It returns true or false.

Example Code:

```
// Returns true if the element exists in the array.
console.log(arr1.includes(40));
// Returns false if the element does not exist in the array.
console.log(arr1.includes(11));
```

4. join() Method

The join() method is used to combine all elements of an array into a single string using a specified separator.

Example Code:

```
let arr2 = [1, 2, 3, 4, 5];

// Joins all array elements into a string using "*" as the separator.

let joinedVal = arr2.join("*");

console.log(joinedVal);
```

5. toString() Method

The toString() method is used to convert an array into a comma-separated string.

Example Code:

```
let arr3 = [10, 20, 30, 40, 50];

// Converts the array into a comma-separated string.

let stringVal = arr3.toString();

console.log(stringVal);
```

Module 40 - Array Methods (indexOf & lastIndexOf)

1. indexOf() Method

The indexOf() method is used to find the first occurrence of a specified element in an array by searching from left to right. It returns the index if found; otherwise, it returns -1.

Example Code:

```
// Search Left to Right
let arr = [10, 20, 30, 40, 50, 10];
// Searches for 20 starting from index 0.
// Returns the index of the first matching element.
let newIndex = arr.indexOf(20, 0);
console.log(newIndex);
```

2. lastIndexOf() Method

The lastIndexOf() method is used to find the last occurrence of a specified element in an array by searching from right to left. It returns the index if found; otherwise, it returns -1.

Example Code:

```
// Search Right to Left
let arr1 = [10, 20, 30, 20, 40, 50, 10];
// Searches for 20 starting from index 0 and moves right-to-left.
// Returns the index of the last matching element within the search range.
let findIndexFromLast = arr1.lastIndexOf(20, 0);
console.log(findIndexFromLast);
```

Module 41 - Array Higher Order Methods (forEach vs map)

1. What are Array Higher-Order Methods?

Array Higher-Order Methods are built-in JavaScript methods that take a callback function as an argument to perform operations on array elements.

2. What is forEach()?

The forEach() method is used to iterate over each element of an array and perform an operation. It does not return a new array.

3. What is map()?

The map() method is used to iterate over each element of an array, perform an operation, and return a new array with the modified values.

Example Code:

I. forEach() Method

```
let fruits = ["Apple", "WaterMelon", "MuskMelon", "Banana"];
// fruits.forEach(printFruit);
// function printFruit(currentElement, index, totalArray){
//     console.log(currentElement, index, totalArray);
// }
// Executes the callback for each array element.
// forEach() always returns undefined.
let newArr = fruits.forEach((cElement)=>{
    console.log(cElement.toUpperCase());
    return cElement;
});
console.log(newArr);
```

II. map() Method

```
// fruits.map(function(currentEle, index, totalArr){
//     console.log(currentEle, index, totalArr);
// })
// Creates and returns a new array.
let newArr1 = fruits.map((cElement, index)=>{
   return {id : index + 1, fruit : cElement};
});
console.log(newArr1);
```

III. Chaining Method

```
// let newArr3 = fruits.forEach(cEle => cEle.toUpperCase()).sort().fill("123");
// console.log(newArr3); // TypeError: Cannot read properties of undefined (reading 'sort')
// map() returns a new array, so methods can be chained.
let newArr2 = fruits.map(cEle => cEle.toUpperCase()).sort().fill("123");
console.log(newArr2);
```

IV. Conditional Based Statement

```
// Returns a new array of boolean values.
let newArr4 = fruits.map((cEle) => {return cEle == "Apple"});
console.log(newArr4);
// Executes the callback for each element but does not return a new array.
let newArr5 = fruits.forEach(val => console.log(val == "Apple"));
```

Module 42 - Array Higher Order Methods (filter vs find)

1. What is filter()?

The filter() method is used to return a new array containing all elements that satisfy a given condition. If no element satisfies the condition, it returns an empty array [].

2. What is find()?

The find() method is used to return the first element that satisfies a given condition. If no element is found, it returns undefined.

Example Code:

```
let employees = [
    {empName : "Kesavan", salary : 150000},
    {empName : "Murugesan", salary : 100000},
    {empName : "John", salary : 120000}
]

// filter() Method
// Returns a new array containing employees whose salary is greater than 110000.
let filterData = employees.filter(val => val.salary > 110000);
console.log(filterData);
// filter() can be chained with other array methods.
let filterData1 = employees.filter(val => val.salary > 110000).fill({id : 1, name1 : "xyz"});
console.log(filterData1);
// Checks every element in the array.
employees.filter(val => console.log(val));

// find() Method
// Returns the first employee whose salary is greater than 100000.
let filterDataByFind = employees.find((val) => {
    return val.salary > 100000;
});
console.log(filterDataByFind);
```

Module 43 - Array Higher Order Methods (sort, some & every)

1. sort() Method

The sort() method is used to arrange the elements of an array in ascending or descending order.

Example Code:

```
// Ascending
// a - b => Positive, a > b => Swapping
// a - b => Negative, a < b => No Swapping
let arr = [10, 5, 100, 30, 6, 2];
// Sorts the array in ascending order.
let newArr = arr.sort((a, b) => { return a - b });
// a - b => 10 - 5 => 5 => Positive => Swap => [5, 10, 100, 30, 6, 2]
// a - b => 10 - 100 => -90 => Negative => No Swap => [5, 10, 100, 30, 6, 2]
// a - b => 100 - 30 => 70 => Positive => Swap => [5, 10, 30, 100, 6, 2]
// a - b => 100 - 6 => 94 => Positive => Swap => [5, 10, 30, 6, 100, 2]
// a - b => 100 - 2 => 98 => Positive => Swap => [5, 10, 30, 6, 2, 100]
// Final Result => [2, 5, 6, 10, 30, 100]
console.log(newArr);

// Descending
// b - a => Positive, b > a => Swapping
// b - a => Negative, b < a => No Swapping
let newArr1 = [10, 20, 30, 1, 4, true, "100"];
// Sorts the array in descending order.
let descendingSort = newArr1.sort((a, b) => { return b - a });
// b - a => 20 - 10 => 10 => Positive => Swap => [20, 10, 30, 1, 4, true, "100"]
// b - a => 30 - 10 => 20 => Positive => Swap => [20, 30, 10, 4, 1, true, "100"]
```

```
// b - a => true - 1 => 1 - 1 => 0 => Equal => No Swap => [20, 30, 10, 4, 1, true, "100"]
// b - a => "100" - true => 100 - 1 => 99 => Positive => Swap => [20, 30, 10, 4, 1, "100", true]
// Final Result => ["100", 30, 20, 10, 4, 1, true]
console.log(descendingSort);
```

2. some() Method

The some() method is used to check whether at least one element in an array satisfies a given condition. It returns true or false.

Example Code:

```
let arr1 = [1, 2, 3, 4, 5];
// Returns true if at least one element satisfies the condition.
let value = arr1.some((ele, index, arr) => {
    return ele % 2 == 0;
});
```

3. every() Method

The every() method is used to check whether all elements in an array satisfy a given condition. It returns true or false.

Example Code:

```
let arr2 = [1, 2, 3, 4, 5];
// Returns true only if all elements satisfy the condition.
let value1 = arr2.every((ele, index, arr) => {
    return ele % 2 == 0;
});
console.log(value, value1);
```

Module 44 - Array Higher Order Methods (reduce Method)

1. What is reduce() Method?

The reduce() method is used to reduce all the elements of an array into a single value by applying a function to each element.

Example Code:

```
// Sum of Array Elements
let arr = [10, 2, 3, 4, 5];
// 'accumulator' stores the accumulated result.
// '0' is the initial value of the accumulator.
let totalVal1 = arr.reduce((accumulator, cElement, index, array) => {
    return accumulator + cElement;
}, 0);
// 1st => accumulator + cElement => 0 + 10 => 10
// 2nd => accumulator + cElement => 10 + 2 => 12
// 3rd => accumulator + cElement => 12 + 3 => 15
// 4th => accumulator + cElement => 15 + 4 => 19
// 5th => accumulator + cElement => 19 + 5 => 24
console.log(totalVal1);

let emloyees = [
    {eName : "Oggy", salary : 10000},
    {eName : "Jack", salary : 20000},
    {eName : "Bob", salary : 30000},
    {eName : "Olivia", salary : 40000}
];
// Calculates the total salary of all employees.
let calcToSalary = emloyees.reduce((acc, cElement) => {
    return acc + cElement.salary;
}, 0);
console.log(calcToSalary);
```

Module 45 - String Methods

1. What are String Methods?

String methods are built-in JavaScript functions used to perform different operations on strings, such as searching, extracting, modifying, or converting string values.

2. charAt() Method

```
let str = "Hello World";
// Returns the character at the specified index.
console.log(str.charAt(str.length - 1));
```

3. charCodeAt() Method

```
let str1 = "Javascript";
// Returns the Unicode value of the character at the specified index.
console.log(str1.charCodeAt(6));
```

4. concat() Method

```
// Combines two or more strings and returns a new string.
let newStr = str.concat(" ", str1);
console.log(newStr);
```

5. includes() Method

```
let str2 = "Single Threaded";
// Checks whether the specified text exists in the string.
console.log(str2.includes("j"));
```

6. indexOf() Method

```
let newStr1 = "Kesavan";
// Returns the index of the first occurrence of the specified character.
console.log(newStr1.indexOf("a", 4));
```

7. lastIndexOf() Method

```
let newStr2 = "Kesavan";
// Returns the index of the last occurrence of the specified character.
console.log(newStr2.lastIndexOf("a", 4));
```

8. repeat() Method

```
let newStr3 = "Javascript";
// Repeats the string the specified number of times.
console.log(newStr3.repeat(3));
```

9. replace() Method / replaceAll() Method

```
// replace() Method / replaceAll() Method
let str3 = "Js is a Script language - Js";
// Replaces the first matching occurrence.
console.log(str3.replace("Js", "Javascript"));
// Replaces all matching occurrences.
console.log(str3.replaceAll("Js", "Javascript"));
```

10. slice() Method

```
let newStr4 = "Single Thread";
// Extracts a portion of the string from the specified index.
console.log(newStr4.slice(3));
// Extracts characters from index 3 up to, but not including, index 8.
console.log(newStr4.slice(3, 8));
// Extracts a portion using negative indexes from the end.
console.log(newStr4.slice(-3, -1));
```

11. substring() Method / substr() Method

```
let newStr5 = "Single Thread";
// Extracts characters from the specified index to the end.
console.log(newStr5.substring(3));
// Extracts characters from index 3 up to, but not including, index 8.
console.log(newStr5.substring(3, 8));
// Negative values are treated as 0 by substring().
console.log(newStr5.substring(-3));
// If start is greater than end, substring() swaps the two values.
console.log(newStr5.substring(8, 0));
```

12. split() Method

```
let words = "My Name is Iron-Man";
// Splits the string into an array using space as the separator.
console.log(words.split(" "));
// Splits using space and limits the result to 3 elements.
console.log(words.split(" ", 3));
// Splits the string using "-" as the separator.
console.log(words.split("-"));
// Splits using "-" and limits the result to 1 element.
console.log(words.split("-", 1));
```

13. startsWith() Method

```
let words1 = "My Name is Iron-Man";
// Checks whether the string starts with the specified value.
console.log(words1.startsWith("M"));
// Checks whether "N" starts at index 3.
console.log(words1.startsWith("N", 3));
// Checks whether "N" starts at index 4.
console.log(words1.startsWith("N", 4));
// Checks whether "Name" starts at index 3.
console.log(words1.startsWith("Name", 3));
```

14. endsWith() Method

```
let words2 = "My Name is Iron-Man";
// Checks whether the string ends with the specified value.
console.log(words2.endsWith("n"));
// Checks whether the string ends with "n" at the specified length.
console.log(words2.endsWith("n", 5));
// Checks whether the string ends with "n" using the full string length.
console.log(words2.endsWith("n", words2.length));
// Checks whether the first 14 characters end with "Iron".
console.log(words2.endsWith("Iron", words2.length - 4));
// Checks whether the first 14 characters end with "Iron".
console.log(words2.endsWith("Iron", words2.length - 4));
```

15. toLowerCase() Method

```
let words3 = "My Name is Iron-Man";
// Converts all characters to lowercase.
console.log(words3.toLowerCase());
```

16. toUpperCase() Method

```
let words4 = "My Name is Iron-Man";
// Converts all characters to uppercase.
console.log(words4.toUpperCase());
```

17. trim() Method

```
let words5 = "   My Name is Iron-Man   ";
// Removes whitespace from both the beginning and end.
console.log(words5.trim());
```

18. trimStart() Method / trimLeft() Method

```
let words6 = "   My Name is Iron-Man   ";
// Removes whitespace from the beginning.
console.log(words6.trimStart());
console.log(words6.trimLeft());
```

19. trimEnd() Method / trimRight() Method

```
let words7 = "   My Name is Iron-Man   ";
// Removes whitespace from the end.
console.log(words7.trimEnd());
console.log(words7.trimRight());
```

Module 46 - Math & Date Objects

1. What is Math Object?

The Math object is a built-in JavaScript object that provides properties and methods for performing mathematical calculations.

Example Code:

I. Math.abs(x)

```
// Returns the absolute (positive) value of a number.
console.log(Math.abs(0));
console.log(Math.abs(-10));
```

II. Math.sign(x)

```
// Returns the sign of a number: -1 for negative, 0 for zero, and 1 for positive.
console.log(Math.sign(-10));
console.log(Math.sign(0));
console.log(Math.sign(10));
```

III. Math.sqrt(x)

```
// Returns the square root of a number.
console.log(Math.sqrt(25));
```

IV. Math.cbrt(x)

```
// Returns the cube root of a number.
console.log(Math.cbrt(27));
```

V. Math.pow(base, exponent)

```
// Returns the base raised to the given exponent.
console.log(Math.pow(6, 3));
```

VI. Math.min(...values)

```
// Returns the smallest value from the given values.
let arr = [1, 2, 3, 4, 5];
console.log(Math.min(...arr, 10, 15, 0, -1));
```

VII. Math.max(...values)

```
// Returns the largest value from the given values.
console.log(Math.max(...arr, 100, 30));
```

VIII. Math.random()

```
// Returns a random decimal number from 0 (inclusive) to 1 (exclusive).
let randomNum = Math.random() * 100;
console.log(randomNum.toFixed(3));
```

IX. Math.ceil(x)

```
// Rounds a number up to the nearest integer.
console.log(Math.ceil(2.1));
```

X. Math.floor(x)

```
// Rounds a number down to the nearest integer.
console.log(Math.floor(2.99));
```

XI. Math.round(x)

```
// Rounds a number to the nearest integer.
console.log(Math.round(2.49));
console.log(Math.round(2.51));
```

XII. Math.trunc(x)

```
 // Removes the decimal part and returns only the integer part.
console.log(Math.trunc(12.999999));
```

2. What is Date Object?

The Date object is a built-in JavaScript object used to create, access, and manipulate dates and times.

Example Code:

I. Date.getFullYear()

```
// Creates a Date object with the current date and time.
let date = new Date();
console.log(date);
// Returns the year from the Date object.
console.log(date.getFullYear());
```

II. Date.getMonth()

```
// Returns the month as a number from 0 to 11, so +1 gives the actual month number.
console.log(date.getMonth() + 1);
```

III. Date.getDate()

```
// Returns the day of the month.
console.log(date.getDate());
```

IV. Date.getHours()

```
// Returns the hour from the Date object.
console.log(date.getHours());
```

V. Date.getMinutes()

```
// Returns the minutes from the Date object.
console.log(date.getMinutes());
```

VI. Date.getSeconds()

```
// Returns the seconds from the Date object.
console.log(date.getSeconds());
```

VII. Date.setFullYear(year, month, day)

```
let date1 = new Date();
// Sets the year, month, and day.
// Month is zero-based: 7 represents August.
date1.setFullYear(2002, 7, 15);
console.log(date1);
```

VIII. Date.setMonth(month, day)

```
let date2 = new Date();
// Sets the month and day.
// Month is zero-based, so 12 represents January of the next year.
date2.setMonth(12, 15);
console.log(date2.toLocaleDateString());
console.log(date2.toDateString());
console.log(date2.toLocaleString());
console.log(date2.toLocaleTimeString());
```

IX. Date.setDate(day)

```
let date3 = new Date();
// Sets the day of the month.
date3.setDate(25);
console.log(date3);
```

X. Date.setHours(hours, minute, second, millisecond)

```
let date4 = new Date();
// Sets the hour, minute, second, and millisecond.
date4.setHours(10, 30, 45, 500);
console.log(date4);
```

XI. Date.setMinutes(minute, second, millisecond)

```
let date5 = new Date();
// Sets the minute, second, and millisecond.
date5.setMinutes(45, 20, 250);
console.log(date5);
```

XII. Date.setSeconds(second, millisecond)

```
let date6 = new Date();
// Sets the second and millisecond.
date6.setSeconds(50, 800);
console.log(date6);
```

XIII. Date.setMilliseconds(millisecond)

```
let date7 = new Date();
// Sets the millisecond.
date7.setMilliseconds(999);
console.log(date7);
```

Module 47 - Object Methods & Prototypal Inheritance

1. What are Object Methods?

Object methods are built-in JavaScript functions used to perform different operations on objects, such as creating, copying, accessing, converting, or freezing objects.

2. What is Prototypal Inheritance?

Prototypal Inheritance is a JavaScript mechanism where an object can access properties and methods from another object through its prototype.

Index.js Code:

I. Object.create()

```
let person = { pName: "Johny", age: 30 };
console.log(person);
// Creates a new object whose prototype is 'person'.
let newObj = Object.create(person);
newObj.location = "Tamilnadu";
// Shows the prototype of newObj.
console.log(newObj.proto);
// Gets the prototype of newObj.
console.log(Object.getPrototypeOf(newObj));
// 'age' is inherited from the person object.
console.log(newObj.age);
console.log(newObj);
```

II. Object.assign()

```
let person1 = { id: "QUI123", name1: "Kesavan" };
// Copies the properties into person1 and returns the modified object.
let newObj1 = Object.assign(person1, { role: "Frontend Developer", Salary: 20000 });
console.log(person1); console.log(newObj1);
```

III. Object.entries()

```
let employees = { eName: "Murugan", eRole: "Backend Developer" };
// Converts object properties into an array of key-value pairs.
let multiArr = Object.entries(employees);
multiArr.push(["Name", "Murugesen"]);
console.log(multiArr);
```

IV. Object.fromEntries()

```
// Converts key-value pairs back into an object.
let normalObj = Object.fromEntries(multiArr);
console.log(normalObj);
```

V. Object.keys()

```
// Returns an array containing all property names (keys).
let onlyKeys = Object.keys(normalObj);
console.log(onlyKeys);
```

VI. Object.values()

```
// Returns an array containing all property values.
let onlyValues = Object.values(normalObj);
console.log(onlyValues);
```

VII. Object.freeze()

```
let newObj2 = { id: 1 };
// Prevents adding, deleting, or modifying properties.
Object.freeze(newObj2);
newObj2.name1 = "Javascript";
newObj2.id = "QUI123";
console.log(newObj2);
```

VIII. Object.isFrozen()

```
// Checks whether the object is frozen.
console.log(Object.isFrozen(newObj2));
console.log(Object.isFrozen(newObj1));
console.log(Object.isFrozen(normalObj));
```

Index.html Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Object Methods, Prototypal Inheritance</title>
</head>
<body>
    <h1>Object Methods - Prototypal Inheritance</h1>
    <script src="index.js"></script>
</body>
</html>
```

Module 48 - Function Methods (call, apply & bind)

1. What are Function Methods?

Function methods are built-in JavaScript methods used to control the value of **'this'** when calling a function.

2. call() Method

The call() method calls a function immediately with a specified **'this'** value and passes arguments individually.

3. apply() Method

The apply() method calls a function immediately with a specified **'this'** value and passes arguments as an array.

4. bind() Method

The bind() method creates a new function with a specified **'this'** value and arguments, which can be called later.

Example Code:

```
let person = {
   pFName: "Kesavan",
   pLName: "Murugesan"
};

let person1 = {
   pId: "UQI123",
   pFName: "Nanda",
   pLName: "Kumar",
    introYourself: function(a, b) {
      // 'this' refers to the object provided while calling the function.
      console.log(this.pFName + " " + this.pLName + (a + b));
      // Returns the sum of a and b.
      return (a + b);
   }
};
```

```
// call() Method
// Calls the function immediately.
// 'person' becomes the value of 'this'.
// Arguments are passed individually.
person1.introYourself.call(person, 10, 20);

// apply() Method
// Calls the function immediately.
// 'person' becomes the value of 'this'.
// Arguments are passed as an array.
person1.introYourself.apply(person, [100, 200]);

// bind() Method
// Creates a new function with 'person' as 'this'.
// The function is not executed immediately.
let newFun = person1.introYourself.bind(person, 500, 300);
// Executes the function later.
console.log(newFun());
```

Module 49 - Deep Copy vs Shallow Copy, Call by Value & Reference

1. What is Call by Value?

Call by Value means a copy of the value is passed or assigned to another variable, so changing one variable does not affect the other variable.

Example Code:

```
// Stack - to store primitive values
// Heap - to store non-primitive values (array, object, function)

/*  Call By Value

   Stack (Before Modification):

   | Address  | Variable  | Value  |
   |----------|-----------|--------|
   | 0x100    |    x      |  10    |
   | 0x104    |    y      | x = 10 |

   Stack (After Modification):

   | Address  | Variable  | Value  |
   |----------|-----------|--------|
   | 0x100    |    x      |  10    |
   | 0x104    |    y      |  20    |
*/
let x = 10;
// The value of x is copied into y.
let y = x;
// Changing y does not affect x because y has its own copied value.
y = 20;
console.log(x, y);
```

2. What is Call by Reference?

Call by Reference means multiple variables refer to the same object or array in memory, so changing the object or array through one variable also affects the other variable.

Example Code:

```
/*  Call By Reference

    Stack Memory:

    | Address  | Variable  | Reference |
    |----------|-----------|-------------|
    | 0x100    | obj1      | 0x001       |
    | 0x104    | arr1      | 0x002       |
    | 0x108    | obj2      | 0x001       |
    | 0x112    | arr2      | 0x002       |

    Heap Memory:

    | Address  | Object                  |
    |----------|-------------------------|
    | 0x001    | {name : 'Kesavan'} |
    | 0x002    | [1, 2, 3]               |
*/
let obj1 = {
    name: "Kesavan"
};
let arr1 = [1, 2, 3];
// obj2 receives the same reference as obj1.
let obj2 = obj1;
// arr2 receives the same reference as arr1.
let arr2 = arr1;
```

```
// Changes the object through the obj1 reference.
obj1.name = "Murugesan";
obj1.role = "Developer";
// Changes the array through the arr1 reference.
arr1[0] = "One";
console.log(obj1, obj2);
console.log(arr1, arr2);
```

3. What is Shallow Copy?

Shallow Copy creates a new object or array, but nested objects or arrays inside it still share the same reference with the original.

4. What is Deep Copy?

Deep Copy creates a completely independent copy, including all nested objects and arrays, so changes in the copy do not affect the original.

Example Code:

```
// Primitive Copy
let a = 10;
let b = a;
a = 20;
console.log(a, b);

// Object - Shallow Copy
let obj1 = {
   name1 : "Murugesan",
   role : "Developer"
};
let obj2 = {...obj1};
obj2.name1 = "Kesavan";
console.log(obj1, obj2);
```

```
// Nested Object
let person = {
    name1 : "Johny",
    role : "Youtuber",
    hobbies : {
        cricket : "T20 Player",
        football : "60 mins"
    }
};
// Shallow Copy with Nested Object
let person2 = {...person, hobbies : {...person.hobbies}};
// Deep Copy with Nested Object
let newObj = JSON.parse(JSON.stringify(person));
person.role = "Video Editor";
person.hobbies.cricket = "ODI Player";
console.log(person, person2);
console.log(newObj);

// Nested Array
let Arr = [1, 2, 3, [4, 5, 6]];
// Shallow Copy with Nested Array
let Arr1 = [...Arr];
// Deep Copy with Nested Array
let newArr = JSON.parse(JSON.stringify(Arr));
Arr[0] = "one";
Arr[3][0] = "Four";
newArr[3][1] = "Five";
console.log(Arr, Arr1);
console.log(newArr);
```

Module 50 - Synchronous vs Asynchronous

1. What is Synchronous?

Synchronous execution means tasks are executed one after another in order, and the next task waits until the current task is completed.

2. What is Asynchronous?

Asynchronous execution means tasks can start and continue without waiting for another task to finish, allowing other tasks to execute while waiting.

Example Code:

```
function f1(){
   console.log("First");
}
function f2(){
   console.log("Second");
}
function f3(){
   console.log("Third");
}
f1();                    // Synchronous: executes immediately.
setTimeout(f2, 2000); // Asynchronous: executes after at least 2 seconds.
f3();                    // Synchronous: executes immediately after f1().

/*
   Call Stack --> Web API --> Call Back Queue --> Event Loop

   Event Loop:

   1. Micro Task Queue - First Priority
      - Promise.then()

   2. Macro Task Queue - Second Priority
      - setTimeOut
      - setInterval
*/
```

- **Call Stack** → Executes JavaScript functions one by one.
- **Web API** → Handles time-consuming tasks in the background.
- **Callback Queue** → Holds completed asynchronous callbacks waiting to run.
- **Event Loop** → Checks the Call Stack and moves callbacks to it when it is empty.
- **Microtask Queue** → A queue that stores high-priority asynchronous tasks that run before the callback queue.
- **Macro Task Queue** → A queue that stores asynchronous tasks that run after the microtask queue.

Module 51 – Promises

1. What is a Promise?

A Promise is a JavaScript object used to handle an asynchronous operation and represent its future result, which can be either successful or failed.

Promise States:

- **Pending** → Operation is still in progress.
- **Fulfilled** → Operation completed successfully.
- **Rejected** → Operation failed.

Promise Methods:

- **.then()** → Handles a successful result.
- **.catch()** → Handles an error or rejected result.
- **.finally()** → Executes after the Promise is completed, whether successful or failed.

Example Code:

```
let newPromise = new Promise((resolve, reject) => {
    let dataReceived = true;
    if(dataReceived){
        resolve("Data Received");
    }
    else{
        reject("Data Not Received");
    }
});

newPromise
    .then((message) => {
        console.log("Success: " + message);
        return "Next Success " + message;
    })
    .then((nextMessage) => {
        console.log(nextMessage);
    })
    .catch((error) => {
        console.log("Failure: " + error);
    })
    .finally(() => {
        console.log("End");
    });
```

Module 52 - async & await

1. What is async?

async is a keyword used before a function to make it return a Promise and allow the use of await inside the function.

2. What is await?

await is a keyword used inside an async function to wait for a Promise to complete before continuing with the next statement.

Example Code:

```
let newPromise = new Promise((fullFilled, failure) => {
    let dataReceived = true;
    if(dataReceived){
        fullFilled("Data Fetched Successfully");
    }
    else{
        // failure("Data Not Found");
        throw new Error("Search Proper Data");
    }
});
async function executePromise(){
    try{
        let message = await newPromise;
        let newMessage = await newPromise;
        console.log(message);
        console.log(`Next Message: ${newMessage}`);
    }
    catch(error){
        console.log(error.message);
    }
    finally{
        console.log("End");
    }
}
executePromise();
console.log("Last");
```

3. What is Exception Handling?

Exception Handling is a mechanism used to detect, handle, and manage errors during program execution without stopping the entire program.

- **try** is used to write code that may cause an error.
- **catch** is used to handle the error that occurs inside the **try** block.
- **finally** is used to execute code after **try** and catch, whether an error occurs or not.
- **throw** is used to manually create and send an error to the **catch** block.

Example Code:

```
try {
   let age = 15;
   if (age < 18) {
      throw new Error("You are not eligible.");
   }
   console.log("You are eligible.");
}
catch (error) {
   console.log("Error: " + error.message);
}
finally {
   console.log("Execution completed.");
}
```

Module 53 - Fetch API

1. What is Fetch API?

- Fetch API is a built-in JavaScript API used to send HTTP requests and receive data from a server.
- It returns a Promise that resolves to the response of the request.
- By default, fetch() uses the GET HTTP method.

Example Code:

I. Fetch API using .then() and .catch()

```
// Sends a GET request to the API.
fetch("https://fakestoreapi.com/users/abcd")
.then((response) => {
   // Checks whether the HTTP response was successful.
   if(!response.ok){
      throw new Error("Data Not Found");
   }
   else{
      // Converts the response body from JSON into JavaScript data.
      return response.json();
   }
})
.then((data) => {
   // Receives and displays the converted data.
   console.log(data);
})
.catch((error) => {
   // Handles any error from the request or Promise chain.
   console.log(error.message);
})
```

II. Fetch API using async & await

```
async function fetchData(){
    try{
        // Sends a GET request and waits for the response.
        let response = await fetch("https://fakestoreapi.com/users");
        // Checks whether the HTTP response was successful.
        if(!response.ok){
            throw new Error("Data Not Found");
        }
        else{
            // Converts the response body from JSON into JavaScript data.
            let data = await response.json();
            // Displays the first user from the returned array.
            console.log(data[0]);
        }
    }
    catch(error){
        // Handles errors from fetch or the thrown error.
        console.log(error.message);
    }
}
// Calls the async function.
fetchData();
```

2. HTTP Methods in REST API

- **GET** → Used to retrieve data from a server.
- **POST** → Used to create/send new data on a server.
- **PUT** → Used to replace or update existing data on a server
- **DELETE** → Used to delete data from a server.
- **HEAD** → Used to get response headers without the response body.
- **OPTIONS** → Used to check which HTTP methods or options are supported by a server.
- **PATCH** → Used to partially update existing data on a server.
- **TRACE** → Used to diagnose the path of a request between the client and server.
- **CONNECT** → Used to establish a network connection to a server, commonly for a proxy tunnel.

Module 54 – Modules

1. What are Modules?

Modules are separate JavaScript files used to organize code into smaller, reusable parts. They allow us to share code between files using export and import.

- **export** → Sends code from a module.
- **import** → Brings exported code into another module.

2. Types of Export

- **Named Export** → Exports specific variables, functions, or classes using their names.
- **Default Export** → Exports one main value from a module and can be imported with any name.

Example Code:

I. **Using .mjs Extension** → .mjs files are treated as JavaScript modules by default, so they can directly use import and export.

Main.mjs:

```
// import { loginData as newPerson } from "./loginPage.mjs";
// import PersonObject from "./signUpPage.mjs";
import { newEmployeeId, loginInfo } from './loginPage.mjs';
import signUpInfo from './signUpPage.mjs';
function application(){
    // console.log(newPerson);
    // console.log(PersonObject);
    console.log(newEmployeeId);
    loginInfo();
    signUpInfo();
}
application();
```

LoginPage.mjs:

```
// Named Export
// export let loginData = {
//     uName : "Deveeswar",
//     role : "Developer"
// }
export let newEmployeeId = "UQI12345";
let loginData = {
    uName : "Deveeswar",
    role : "Developer"
};
export function loginInfo(){
    console.log(loginData);
}
```

SignUpPage.mjs

```
// Default Export
let signUpData = {
    uName : "Johny",
    uEmail : "abc@gmail.com",
    age : 45
};
// export default signUpData;
export default function signUpInfo(){
    console.log(signUpData);
}
```

II. **Using .js with package.json** → .js file can be treated as a module by setting "type": "module" in package.json.

Index.js

```
import { products } from "./wishlist.js";
function indexFile(){
   console.log(products);
}
indexFile();
```

Wishlist.js

```
export let products = [
   {id : 1, pName : "Mobile"}
];
```

Package.json

```
{
 "name": "javascript",
 "version": "1.0.0",
 "description": "Javascript Series",
 "license": "ISC",
 "author": "Deveeswar",
 "type": "module",
 "main": "index.js",
 "scripts": {
  "test": "echo \"Error: no test specified\" && exit 1"
 }
}
```

III. **Using .js with index.html** → .js file can be treated as a module in HTML by using type="module" in the <script> tag.

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Modules</title>
</head>
<body>
    <h1>Modules in Javascript</h1>
    <!-- type="module" allows import/export in JavaScript -->
    <script type="module" src="products.js"></script>
</body>
</html>
```

Products.js

```
import { products } from "./userData.js";
console.log(products);
```

UserData.js

```
export let products = [
    { id: 1, pName: "Mobile" },
    { id: 2, pName: "Laptop" }
];
```

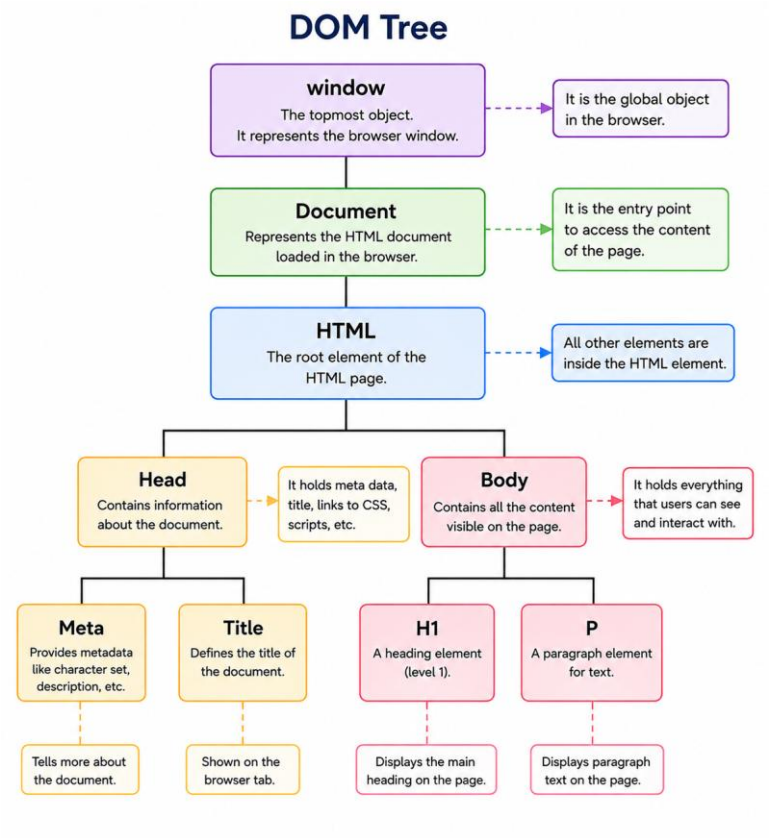
Module 55 - DOM Selecting Elements

1. What is DOM?

DOM (Document Object Model) is a programming interface that represents an HTML document as a tree of objects, allowing JavaScript to access and modify the HTML elements and content.

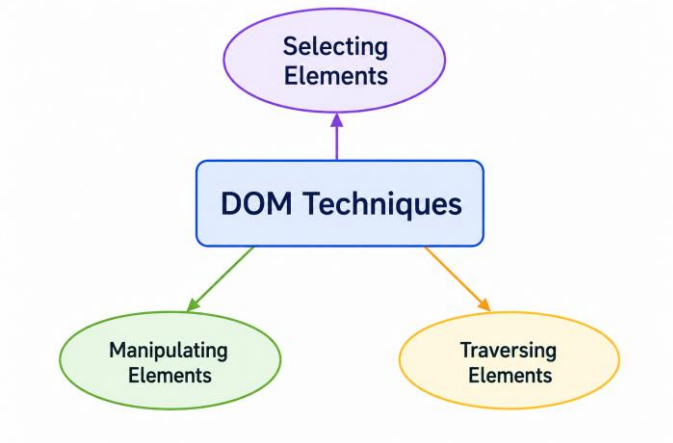
2. What is DOM Tree?

DOM Tree is a hierarchical structure that represents an HTML document as a tree of nodes, where each HTML element is connected as a parent, child, or sibling.



3. What are DOM Techniques?

DOM Techniques are methods used in JavaScript to select, manipulate, and traverse HTML elements in a webpage.



4. What is DOM Selecting Elements?

DOM Selecting Elements is the process of finding and accessing HTML elements from a webpage using JavaScript.

Common DOM Selection Methods

- **getElementsByTagName()** → Selects elements by their HTML tag name.
- **getElementsByClassName()** → Selects elements by their class name.
- **getElementById()** → Selects an element by its ID.
- **getElementsByName()** → Selects elements by their name attribute.
- **querySelector()** → Selects the first matching element using a CSS selector.
- **querySelectorAll()** → Selects all matching elements using a CSS selector.

Example Code:

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>DOM Selecting Elements</title>
</head>
<body>
    <h1>DOM - Document Object Model</h1>
    <h1>Selecting Elements</h1>
    <p class="para1">Content - 1</p>
    <p class="para1">Content - 2</p>
    <p id="uniq-para">Content - 3</p>
    <p id="uniq-para">Content - 4</p>
    <input type="text" name="input-1" placeholder="User Name">
    <script src="index.js"></script>
</body>
</html>
```

Index.js

```
// getElementsByTagName()
let heading = document.getElementsByTagName("h1");
console.log(heading);

// getElementsByClassName()
let para1 = document.getElementsByClassName("para1");
console.log(para1);

// getElementById()
let uniqPara = document.getElementById("uniq-para");
console.log(uniqPara);

// getElementsByName()
let nameAttribute = document.getElementsByName("input-1");
console.log(nameAttribute);

// querySelector()
// let selectOne = document.querySelector("h1");
// let selectOne = document.querySelector(".para1");
let selectOne = document.querySelector("#uniq-para");
// innerHTML gets the HTML content inside the selected element.
console.log(selectOne.innerHTML);

// querySelectorAll()
// let multiElements = document.querySelectorAll("h1");
// let multiElements = document.querySelectorAll(".para1");
let multiElements = document.querySelectorAll("#uniq-para");
console.log(multiElements);
```

Module 56 - DOM Traversing

1. What is DOM Traversing?

DOM Traversing is the process of moving between related HTML elements in the DOM tree, such as parent, child, and sibling elements.

DOM Traversing Properties

I. Parent Properties

- **parentElement** → Returns the parent element.
- **parentNode** → Returns the parent node.

II. Child Properties

- **childElementCount** → Returns the number of child elements.
- **childNodes** → Returns all child nodes, including text nodes.
- **children** → Returns all child elements.
- **firstChild** → Returns the first child node.
- **firstElementChild** → Returns the first child element.
- **lastChild** → Returns the last child node.
- **lastElementChild** → Returns the last child element.

III. Sibling Properties

- **nextSibling** → Returns the next sibling node.
- **nextElementSibling** → Returns the next sibling element.
- **previousSibling** → Returns the previous sibling node.
- **previousElementSibling** → Returns the previous sibling element.

Important Difference: Node properties can include text nodes, while Element properties consider only HTML elements.

Example Code:

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>DOM Traversing</title>
</head>
<body>
    <!-- Parent element containing child elements and text nodes -->
    <div class="parent">
        Javascript
        <div class="child1">Content 1</div>
        Dynamically Typed
        <div class="child2">Content 2</div>
        Programming Language
        <div class="child3">Content 3</div>
        Single Threaded
    </div>
    <!-- Calls the function when the button is clicked -->
    <button onclick="traversingParent()">Parent Properties</button>
    <!-- Calls the function when the button is clicked -->
    <button onclick="selectChild()">Child Properties</button>
    <!-- Calls the function when the button is double-clicked -->
    <button ondblclick="selectSiblings()">Double Click for Siblings Properties</button>
    <script src="index.js"></script>
</body>
</html>
```

Index.js

```
// Parent Properties
function traversingParent(){
    let getParent = document.querySelector(".child1");
    console.log(getParent.parentElement);
    console.log(getParent.parentNode);
    let getParent1 = document.querySelector("html");
    console.log(getParent1.parentElement);
    console.log(getParent1.parentNode);
}

// Node ---> Element Node, Text Node, Attribute Node, Document (Root Node)

// Child Properties
function selectChild(){
    let getElementByClass = document.querySelector(".parent");
    console.log(getElementByClass.childElementCount);
    console.log(getElementByClass.childNodes);
    console.log(getElementByClass.children);
    console.log(getElementByClass.firstChild);
    console.log(getElementByClass.firstElementChild);
    console.log(getElementByClass.lastChild);
    console.log(getElementByClass.lastElementChild);
}

// Sibling Properties
function selectSiblings(){
    let child1 = document.querySelector(".child1");
    console.log(child1.nextSibling);
    console.log(child1.nextElementSibling);
    console.log(child1.previousSibling);
    console.log(child1.previousElementSibling);
}
```

Module 57 - DOM Manipulation

1. What is DOM Manipulation?

DOM Manipulation is the process of creating, changing, adding, replacing, or removing HTML elements and their content using JavaScript.

DOM Manipulation Methods

- **createElement()** → Creates a new HTML element.
- **innerText** → Gets or changes the visible text of an element.
- **innerHTML** → Gets or changes the HTML content inside an element.
- **textContent** → Gets or changes all text content inside an element.
- **append()** → Adds content or elements at the end.
- **appendChild()** → Adds an element as the last child.
- **prepend()** → Adds content or elements at the beginning.
- **insertBefore()** → Inserts an element before a specified child.
- **replaceChild()** → Replaces an existing child with another element.
- **removeChild()** → Removes a specified child element.
- **remove()** → Removes the selected element itself.
- **style** → Changes the CSS styles of an element.

Example Code:

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>DOM Manipulation</title>
    <style>
        .container{
            background-color: red;
            width: 400px;
            margin: auto;
            padding: 10px;
            box-shadow: 10px 10px 3px black;
        }
    </style>
</head>
```

```
<body>
    <!-- Container holding the input, button, and list -->
    <div class="container">
        <!-- Takes input from the user -->
        <input type="text" placeholder="Enter Input">
        <!-- Calls manipulateElements() when clicked -->
        <button onclick="manipulateElements()">Click Me</button>
        <!-- Existing ordered list -->
        <ol>
            <li>item-1</li>
            <li>item-2</li>
            <li>item-3</li>
        </ol>
    </div>
    <script src="index.js"></script>
</body>
</html>
```

Index.js

```
function manipulateElements(){
    // Create li element
    let newListElement = document.createElement("li");

    // Add text
    // newListElement.innerText = "<a>Link</a>Item-4";

    // Add HTML
    // newListElement.innerHTML = "<a>Link</a>Item-5";

    // Add plain text
    // newListElement.textContent = "<a>Link</a>Item-6";

    // Select input
    let input = document.querySelector("input");

    // Get input value
    // console.log(input.value);
```

```
    // Set input value as text
    newListElement.innerText = input.value;

    // Select ordered list
    let orderList = document.querySelector("ol");

    // Add at end
    // orderList.append("Text Node", newListElement);

    // Insert before element
    // orderList.insertBefore(newListElement, orderList.children[2]);

    // Replace element
    // orderList.replaceChild(newListElement, orderList.children[2]);

    // Remove child
    // orderList.removeChild(orderList.children[0]);

    // Remove element
    // orderList.remove();

    // Add at beginning
    // orderList.prepend("Text Node", newListElement);

    // Change text color
    newListElement.style.color = "yellow";

    // Add text shadow
    newListElement.style.textShadow = "10px 10px 1px white";

    // Add element at end
    orderList.appendChild(newListElement);
}
```

Module 58 - Event Handler vs Event Listener

1. What is Event Handler?

- An Event Handler is a function assigned to an event that runs when that event occurs.
- Only one function can be assigned at a time; assigning another replaces the previous one.

2. What is Event Listener?

- An Event Listener is a method that waits for a specific event and runs a function when that event occurs.
- Multiple functions can be attached to the same event.

Example Code:

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Event Handler vs Event Listener</title>
</head>
<body>
    <h1>Event Handler vs Event Listener</h1>
    <img src="img1.jpeg" alt="img1.jpeg">
    <p>I'm a Butterfly</p>
    <button id="listener">Listener</button>
    <button id="handler">Handler</button>
    <script src="index.js"></script>
</body>
</html>
```

Index.js

```
let button1 = document.getElementById("listener");
let button2 = document.getElementById("handler");

// Event Listener
button1.addEventListener("click", function(){
    console.log("First Listener");
});
button1.addEventListener("click", function(){
    console.log("Second Listener");
});
button1.addEventListener("click", function(){
    console.log("Third Listener");
});

// Event Handler
button2.onclick = function(){
    console.log("First Handler");
}
button2.onclick = function(){
    console.log("Second Handler");
}

let image = document.querySelector("img");
let para = document.querySelector("p");
```

```
// Mouse enters the image
image.addEventListener("mouseover",function(){
    image.src="img2.jpeg";
    para.innerText = "I'm a Leaf";
});
// Mouse leaves the image
image.addEventListener("mouseout",function(){
    image.src="img3.jpeg";
    para.innerText = "I'm a Flower";
});
```

Module 59 - Project – Calculator

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="stylesheet" href="index.css">
    <title>Calculator</title>
</head>
<body>
    <!-- Calculator container -->
    <form class="calculator">
        <!-- Display screen -->
        <div class="form-group">
            <input type="text">
        </div>
        <!-- Operators and control buttons -->
        <div class="form-group">
            <button type="button" onclick="addValue('%')">%</button>
            <button type="button" onclick="clearVal()">C</button>
            <button type="button" onclick="deleteCharacter()">DEL</button>
            <button type="button" onclick="addValue('/')">/</button>
        </div>
```

```
<!-- Number buttons -->
<div class="form-group">
    <button type="button" onclick="addValue('1')">1</button>
    <button type="button" onclick="addValue('2')">2</button>
    <button type="button" onclick="addValue('3')">3</button>
    <button type="button" onclick="addValue('+')">+</button>
</div>
<!-- Number buttons -->
<div class="form-group">
    <button type="button" onclick="addValue('4')">4</button>
    <button type="button" onclick="addValue('5')">5</button>
    <button type="button" onclick="addValue('6')">6</button>
    <button type="button" onclick="addValue('-')">-</button>
</div>
<!-- Number buttons -->
<div class="form-group">
    <button type="button" onclick="addValue('7')">7</button>
    <button type="button" onclick="addValue('8')">8</button>
    <button type="button" onclick="addValue('9')">9</button>
    <button type="button" onclick="addValue('*')">*</button>
</div>
<!-- Zero, decimal and result buttons -->
<div class="form-group">
    <button type="button" onclick="addValue('00')">00</button>
    <button type="button" onclick="addValue('0')">0</button>
```

```
            <button type="button" onclick="addValue('.')">.</button>
            <button type="button" onclick="evaluateVal()" class="evaluate">
                =
            </button>
        </div>
    </form>
    <script src="index.js"></script>
</body>
</html>
```

Index.css

```
/* Calculator container */
.calculator{
    width: 30vw;
    height: 70vh;
    border: 1px solid rgb(216, 210, 210);
    margin: auto;
    position: relative;
    top: 100px;
    display: grid;
    gap: 5px;
    padding: 20px;
}
/* Arrange each button row */
.calculator>.form-group{
```

```
    display: flex;
    justify-content: space-between;
    gap: 5px;
}
/* Button styling */
.calculator>.form-group>button{
    width: 25%;
    font-size: 20px;
    border: none;
    border-radius: 5px;
}
/* Button hover effect */
.calculator>.form-group>button:hover{
    background-color: rgb(196, 189, 189);
}
/* Calculator display */
.calculator>.form-group>input{
    width: 100%;
    font-size: 40px;
    border: 1px solid rgb(227, 218, 218);
    text-align: end;
    outline: none;
    border-radius: 5px;
    padding-right: 10px;
}
```

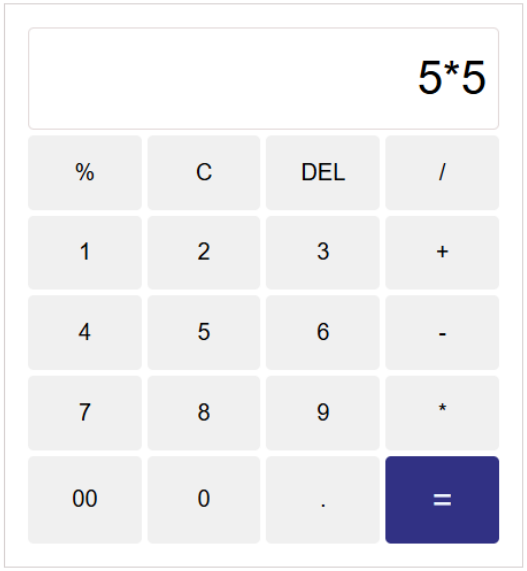
```
/* Equal button */
.calculator>.form-group>.evaluate{
    color: aliceblue;
    background-color: rgb(49, 49, 132);
    font-size: 30px;
}
/* Equal button hover effect */
.calculator>.form-group>.evaluate:hover{
    background-color: rgb(73, 73, 177);
}
```

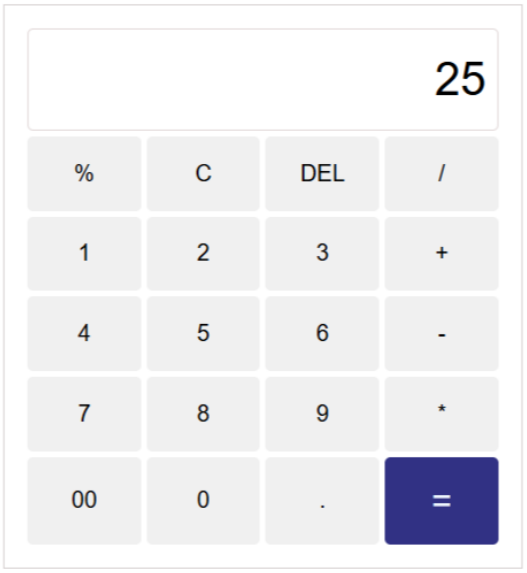
Index.js

```
// Select calculator input
let input = document.querySelector("input");
// Add value to display
function addValue(elementVal){
    input.value += elementVal;
}
// Clear display
function clearVal(){
    input.value = "";
}
// Delete last character
function deleteCharacter(){
    input.value = input.value.slice(0, input.value.length - 1);
```

```
}

// Calculate result

function evaluateVal(){

    input.value = eval(input.value);

}
```

Output:





Module 60 - Project - Form Validation

Index.html

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Form Validation</title>

    <link rel="stylesheet" href="index.css">

</head>

<body>

    <!-- Form heading -->

    <h1>Create Your Account</h1>

    <!-- Validation form -->

    <form id="form-validate">

        <!-- Username field -->

        <div class="form-group">

            <label for="">User Name</label>

            <input type="text" id="userName">

            <span id="uName-error"></span>

        </div>

        <!-- Email field -->

        <div class="form-group">
```

```
        <label for="">Email</label>
        <input type="text" id="email">
        <span id="mail-error"></span>
    </div>
    <!-- Password field -->
    <div class="form-group">
        <label for="">Password</label>
        <input type="password" id="password">
        <span id="password-error"></span>
    </div>
    <!-- Confirm password field -->
    <div class="form-group">
        <label for="">Confirm Password</label>
        <input type="password" id="confirmPassword">
        <span id="cPassword-error"></span>
    </div>
    <!-- Submit button -->
    <button>Submit</button>
  </form>
  <script src="index.js"></script>
</body>
</html>
```

Index.css

```
/* Reset default styles */
*{
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}
/* Page spacing */
body{
    margin-top: 100px;
}
/* Heading style */
h1{
    text-align: center;
    color: rgb(71, 71, 150);
}
/* Form container */
#form-validate{
    border: 1px solid rgb(202, 198, 198);
    width: 30vw;
    min-width: 300px;
    height: 60vh;
    box-shadow: 2px 2px 5px pink;
```

```
    margin: auto;
    display: grid;
    padding: 20px;
}
/* Arrange form fields vertically */
#form-validate .form-group{
    display: flex;
    flex-direction: column;
}
/* Label style */
#form-validate .form-group label{
    font-size: 20px;
    color: rgb(71, 71, 150);
    margin-bottom: 5px;
}
/* Input style */
#form-validate .form-group input{
    height: 30px;
    border: none;
    outline: none;
    border-bottom: 1px solid rgb(211, 207, 207);
    font-size: 15px;
    padding: 5px;
```

```
}
/* Error message style */
#form-validate .form-group span{
    color: red;
    margin-top: 5px;
}
/* Submit button */
#form-validate button{
    height: 35px;
    background-color: rgb(71, 71, 150);
    color: white;
    border-radius: 5px;
    font-size: 20px;
}
```

Index.js

```
// Listen for form submission
document.getElementById("form-validate").addEventListener("submit", function(event){
    // Stop the form from refreshing the page
    event.preventDefault();
    // Get input values
    let userName = document.getElementById("userName").value.trim();
    let email = document.getElementById("email").value.trim();
```

```
let password = document.getElementById("password").value.trim();

let confirmPassword = document.getElementById("confirmPassword").value.trim();

// Select error message elements

let uNameError = document.getElementById("uName-error");

let emailError = document.getElementById("mail-error");

let passError = document.getElementById("password-error");

let cPassError = document.getElementById("cPassword-error");

// Stores whether the form is valid

let isValid = true;

// Username validation pattern

let uNamePattern = /^[A-Za-z]+ [A-Za-z0-9]+$/;

// Email validation pattern

let emailPattern = /^[a-z0-9]+@[a-z]{4,}\.[a-z]{2,}$/;

// Username validation

if(userName === ""){

    uNameError.innerText = "*Name is Required";

    isValid = false;

}

else if(!uNamePattern.test(userName)){

    uNameError.innerText = "*Enter Your Full Name";
```

```
    isValid = false;
}
else if(uNamePattern.test(userName)){
    uNameError.innerText = "";
    isValid = true;
}
// Email validation
if(email === ""){
    emailError.innerText = "*Email is Required";
    isValid = false;
}
else if(!emailPattern.test(email)){
    emailError.innerText = "*Enter a valid email";
    isValid = false;
}
else if(emailPattern.test(email)){
    emailError.innerText = "";
    isValid = true;
}
// Password validation
```

```
if(password === ""){
    passError.innerText = "*Password is Required";
    isValid = false;
}
else if(password.length <= 3 || password.length >= 10){
    passError.innerText = "*Enter a password Character between 3 to 10";
    isValid = false;
}
else if(password.length > 3 || password.length < 10){
    passError.innerText = "";
    isValid = true;
}
// Confirm password validation
if(confirmPassword === ""){
    cPassError.innerText = "*Confirm Password is Required";
    isValid = false;
}
else if(confirmPassword !== password){
    cPassError.innerText = "*Password Not Match";
    isValid = false;
```

```
    }
    else if(confirmPassword === password){
        cPassError.innerText = "";
        isValid = true;
    }
    // Submit only when validation succeeds
    if(isValid){
        alert(`Hi ${userName}, Welcome to Our Website`);
        console.log(userName, email, password);
    }
});
```

Output: